光与朽游戏软件源程序

```
// ============================================================
// EnemyData.cs
// 文件位置: Assets/Scripts/Data/SO/EnemyData.cs
// 用途：单个敌人类型的配置数据（ScriptableObject）
// ============================================================
using UnityEngine;
using LightVsDecay.Core.Pool;
namespace LightVsDecay.Data.SO
{
    /// <summary>
    /// 敌人配置数据 (ScriptableObject)
    /// 每种敌人类型对应一个配置文件
    /// </summary>
    [CreateAssetMenu(fileName = "Enemy_New", menuName = "LightVsDecay/Enemy
Data", order = 1)]
    public class EnemyData : ScriptableObject
    {
        // ──────────────────────────────────────────────
        // 基础信息
        // ──────────────────────────────────────────────
        [Header("基础信息")]
        [Tooltip("敌人类型")]
        public EnemyType type = EnemyType.Slime;
        [Tooltip("显示名称")]
        public string displayName = "粘液";
        [Tooltip("描述")]
        [TextArea(2, 4)]
        public string description = "基础敌人单位";
        // ──────────────────────────────────────────────
        // 战斗属性
        // ──────────────────────────────────────────────
        [Header("战斗属性")]
        [Tooltip("最大生命值")]
        [Min(1f)]
        public float maxHealth = 30f;
        [Tooltip("移动速度")]
        [Min(0.1f)]
        public float moveSpeed = 1.0f;
        [Tooltip("物理质量（影响击退效果）")]
        [Min(0.1f)]
        public float mass = 1.0f;
        [Tooltip("接触伤害（碰撞玩家时造成的伤害）")]
        [Min(0)]
        public int contactDamage = 30;
        [Tooltip("攻击间隔（秒，0 表示只攻击一次如自爆怪）")]
        [Min(0f)]
        public float attackInterval = 1.0f;
        [Tooltip("是否为自爆型（碰撞后自毁）")]
        public bool isSuicide = false;
        [Header("═══ 击退抗性 ═══")]
```

```
[Tooltip("击退抗性 (0=无抗性，1=完全免疫)")]
[Range(0f, 1f)]
public float knockbackResistance = 0f;
// ═══════════════════════════════════════════════
// 行为设置
// ═══════════════════════════════════════════════
[Header("行为设置")]
[Tooltip("敌人行为类型")]
public EnemyBehaviorType behaviorType = EnemyBehaviorType.Chase;
[Header("横穿屏幕设置（仅 CrossScreen 类型有效）")]
[Tooltip("波浪线振幅")]
[Range(0f, 3f)]
public float waveAmplitude = 1.0f;
[Tooltip("波浪线频率")]
[Range(0.5f, 5f)]
public float waveFrequency = 1.5f;
[Tooltip("出界后存活时间（秒）")]
[Range(0f, 3f)]
public float outOfBoundsLifetime = 1.0f;
// ═══════════════════════════════════════════════
// 击退设置
// ═══════════════════════════════════════════════
[Header("击退设置")]
[Tooltip("是否可被击退")]
public bool canBeKnockedBack = true;
[Tooltip("击退力度倍率")]
[Range(0f, 3f)]
public float knockbackMultiplier = 1.0f;
[Tooltip("被击退后的阻力")]
[Range(0f, 10f)]
public float knockbackDrag = 2.0f;
[Tooltip("被击退后的僵直时间")]
[Range(0f, 1f)]
public float knockbackStunDuration = 0.2f;
[Tooltip("僵直期间移动力度倍率")]
[Range(0f, 1f)]
public float knockbackStunMoveMultiplier = 0.3f;
[Header("Drifter 特殊击退")]
[Tooltip("Drifter 偏移角度")]
[Range(0f, 90f)]
public float drifterDeflectionAngle = 45f;
[Tooltip("Drifter 击退力度倍率")]
[Range(1f, 20f)]
public float drifterKnockbackMultiplier = 5.0f;
// 【新增】弹飞状态参数
[Tooltip("最小弹飞时间（秒）- 防止立即恢复移动")]
[Range(0.1f, 2f)]
public float knockbackMinDuration = 0.5f;
[Tooltip("Drifter 最大速度限制")]
[Range(5f, 30f)]
```

```
public float drifterMaxSpeed = 15f;
[Tooltip("弹飞结束速度阈值 - 速度低于此值时恢复移动")]
[Range(0.5f, 10f)]
public float knockbackSpeedThreshold = 2.0f;
// ═══════════════════════════════════════════════
// 视觉设置
// ═══════════════════════════════════════════════
[Header("视觉设置")]
[Tooltip("最小缩放（血量为 0 时）")]
[Range(0.1f, 0.5f)]
public float minScale = 0.3f;
[Tooltip("死亡淡出时长")]
[Range(0.1f, 1f)]
public float deathFadeDuration = 0.3f;
[Header("Shader 参数")]
public float normalFlowSpeed = 0.3f;
public float normalNoiseScale = 1.0f;
public float hitFlowSpeed = 2.0f;
public float hitNoiseScale = 2.0f;
public float wobbleReturnSpeed = 5.0f;
// ═══════════════════════════════════════════════
// 奖励设置
// ═══════════════════════════════════════════════
[Header("奖励设置")]
[Tooltip("击杀获得的经验值")]
[Min(0)]
public int xpReward = 10;
[Tooltip("击杀获得的金币")]
[Min(0)]
public int coinReward = 1;
[Header("宝箱怪特殊掉落")]
[Tooltip("被击中时掉落金币")]
public bool dropCoinOnHit = false;
[Tooltip("每次被击中掉落的金币数")]
[Min(0)]
public int coinPerHit = 1;
[Tooltip("死亡时爆出的金币数量")]
[Min(0)]
public int deathCoinBurst = 0;
[Tooltip("低保经验值（玩家等级<12 时）")]
[Min(0)]
public int lowLevelBonusXP = 0;
[Tooltip("低保触发等级")]
public int lowLevelThreshold = 12;
// ═══════════════════════════════════════════════
// 碰撞行为
// ═══════════════════════════════════════════════
[Header("碰撞行为")]
[Tooltip("撞击玩家后的行为")]
```

```
        public      EnemyCollisionBehavior      collisionBehavior      =
EnemyCollisionBehavior.Suicide;
        [Tooltip("是否为弹跳怪（进入屏幕后会与空气墙碰撞）")]
        public bool isBouncing = false;
        [Tooltip("大怪被弹开时的力度")]
        public float bounceForce = 300f;
        [Tooltip("大怪被弹开后的僵直时间")]
        public float bounceStunDuration = 1.0f;
        // ═══════════════════════════════════════════════
        // 便捷方法
        // ═══════════════════════════════════════════════
        /// <summary>
        /// 是否为小型敌人（根据质量判断）
        /// </summary>
        public bool IsSmallEnemy => mass < 2.0f;
        /// <summary>
        /// 计算击杀所需时间（理论值，基于 100DPS）
        /// </summary>
        public float EstimatedKillTime => maxHealth / 100f;
        /// <summary>
        /// 是否为横穿屏幕类型
        /// </summary>
        public     bool     IsCrossScreen     =>     behaviorType     ==
EnemyBehaviorType.CrossScreen;
    }
    /// <summary>
    /// 敌人碰撞行为枚举
    /// </summary>
    public enum EnemyCollisionBehavior
    {
        /// <summary>自爆（小怪）- 立即销毁，播放特效</summary>
        Suicide,
        /// <summary>反弹（大怪）- 被弹开，进入僵直</summary>
        Bounce,
        /// <summary>微弱反弹（BOSS）- 稍微后退，无僵直</summary>
        WeakBounce,
        /// <summary>无碰撞（宝箱怪）- 不与塔碰撞</summary>
        None
    }
    /// <summary>
    /// 敌人行为类型枚举
    /// </summary>
    public enum EnemyBehaviorType
    {
        /// <summary>追击 - 向玩家移动（默认）</summary>
        Chase,
        /// <summary>横穿屏幕 - 从一侧到另一侧（宝箱怪）</summary>
        CrossScreen
    }
}// ============================================================
```

```
// DrifterSpawnHelper.cs
// 文件位置: Assets/Scripts/Logic/Enemy/DrifterSpawnHelper.cs
// 用途：Drifter 专用生成辅助 - 屏幕内安全区域生成 + 传送门特效 + 缩放动画
// ============================================================
using UnityEngine;
using LightVsDecay.Core;
using LightVsDecay.Core.Pool;
namespace LightVsDecay.Logic.Enemy
{
    /// <summary>
    /// Drifter 生成配置
    /// </summary>
    [System.Serializable]
    public class DrifterSpawnConfig
    {
        [Header("安全区域")]
        [Tooltip("距离塔/护盾的最小安全距离")]
        public float minDistanceFromTower = 6f;
        [Tooltip("距离墙壁的最小距离")]
        public float minDistanceFromWall = 1.5f;
        [Tooltip("只在屏幕上半区生成")]
        public bool spawnInUpperHalf = true;
        [Header("特效时间")]
        [Tooltip("传送门特效持续时间")]
        public float portalEffectDuration = 0.5f;
        [Tooltip("缩放动画持续时间")]
        public float scaleAnimDuration = 0.3f;
        [Header("特效引用（可选）")]
        [Tooltip("传送门特效 Prefab（可为空，使用占位效果）")]
        public GameObject portalEffectPrefab;
    }
    /// <summary>
    /// Drifter 生成辅助器
    /// 单例模式，管理 Drifter 的特殊生成逻辑
    /// </summary>
    public class DrifterSpawnHelper : Singleton<DrifterSpawnHelper>
    {
        // ═══════════════════════════════════════════════════
        // Inspector 配置
        // ═══════════════════════════════════════════════════
        [Header("生成配置")]
        [SerializeField] private DrifterSpawnConfig config = new
DrifterSpawnConfig();
        [Header("塔引用")]
        [Tooltip("玩家塔的 Transform（用于计算安全距离）")]
        [SerializeField] private Transform towerTransform;
        [Header("调试")]
        [SerializeField] private bool showDebugInfo = false;
        [SerializeField] private bool showGizmos = true;
        // ═══════════════════════════════════════════════════
```

```
        // 缓存
        // ═══════════════════════════════════════════════════════
        private int bouncingEnemyLayerIndex;
        // ═══════════════════════════════════════════════════════
        // Unity 生命周期
        // ═══════════════════════════════════════════════════════
        protected override void OnSingletonAwake()
        {
            // 缓存 Layer
            bouncingEnemyLayerIndex                                    =
LayerMask.NameToLayer(GameConstants.BOUNCING_ENEMY_LAYER);
            // 自动查找塔
            if (towerTransform == null)
            {
                var tower = FindObjectOfType<Player.TurretHealth>();
                if (tower != null)
                {
                    towerTransform = tower.transform;
                }
            }
        }
        // ═══════════════════════════════════════════════════════
        // 公共接口
        // ═══════════════════════════════════════════════════════
        /// <summary>
        /// 生成 Drifter（带传送门特效和缩放动画）
        /// </summary>
        /// <param name="onSpawnComplete">生成完成回调（返回 EnemyBlob 实例）
</param>
        public void SpawnDrifter(System.Action<EnemyBlob> onSpawnComplete
= null)
        {
            StartCoroutine(SpawnDrifterCoroutine(onSpawnComplete));
        }
        /// <summary>
        /// 获取安全生成位置
        /// </summary>
        public Vector3 GetSafeSpawnPosition()
        {
            return CalculateSafePosition();
        }
        /// <summary>
        /// 检查位置是否安全
        /// </summary>
        public bool IsPositionSafe(Vector3 position)
        {
            return ValidatePosition(position);
        }
        // ═══════════════════════════════════════════════════════
        // 生成流程
```

```
        // ═══════════════════════════════════════════════
        /// <summary>
        /// Drifter 生成协程
        /// </summary>
        private                                                IEnumerator
SpawnDrifterCoroutine(System.Action<EnemyBlob> onSpawnComplete)
        {
            // 1. 计算安全位置
            Vector3 spawnPos = CalculateSafePosition();
            if (showDebugInfo)
            {
                GameLogger.Log($"[DrifterSpawnHelper] 准备在 {spawnPos} 生
成 Drifter");
            }
            // 2. 播放传送门特效
            GameObject portalEffect = null;
            if (config.portalEffectPrefab != null)
            {
                portalEffect   =   Instantiate(config.portalEffectPrefab,
spawnPos, Quaternion.identity);
            }
            else
            {
                // 占位特效：简单的缩放圆圈（后续替换）
                portalEffect = CreatePlaceholderPortalEffect(spawnPos);
            }
            // 等待特效播放
            yield return new WaitForSeconds(config.portalEffectDuration);
            // 销毁特效
            if (portalEffect != null)
            {
                Destroy(portalEffect);
            }
            // 3. 从对象池生成 Drifter
            if (EnemyPoolManager.Instance == null)
            {
                GameLogger.LogError("[DrifterSpawnHelper]
EnemyPoolManager 不存在！");
                yield break;
            }
            EnemyBlob                          drifter                          =
EnemyPoolManager.Instance.Spawn(EnemyType.Drifter, spawnPos);
            if (drifter == null)
            {
                GameLogger.LogWarning("[DrifterSpawnHelper] Drifter 生成失
败（可能达到上限）");
                yield break;
            }
            // 4. 设置 Layer 为 BouncingEnemy（直接可撞墙）
            drifter.gameObject.layer = bouncingEnemyLayerIndex;
```

```
            // 5. 执行缩放动画
            yield return StartCoroutine(ScaleInAnimation(drifter));
            // 6. 通知 Drifter 已完全入境
            drifter.SetFullyEnteredScreen();
            if (showDebugInfo)
            {
                GameLogger.Log($"[DrifterSpawnHelper] Drifter 生成完成 @
{spawnPos}");
            }
            // 7. 回调
            onSpawnComplete?.Invoke(drifter);
        }
        /// <summary>
        /// 缩放入场动画
        /// </summary>
        private IEnumerator ScaleInAnimation(EnemyBlob enemy)
        {
            if (enemy == null) yield break;
            Transform t = enemy.transform;
            Vector3 targetScale = t.localScale;
            // 从 0 开始
            t.localScale = Vector3.zero;
            float elapsed = 0f;
            float duration = config.scaleAnimDuration;
            while (elapsed < duration)
            {
                elapsed += Time.deltaTime;
                float progress = elapsed / duration;
                // 使用 EaseOutBack 曲线，产生"弹出"效果
                float easedProgress = EaseOutBack(progress);
                t.localScale = targetScale * easedProgress;
                yield return null;
            }
            // 确保最终缩放正确
            t.localScale = targetScale;
        }
        /// <summary>
        /// EaseOutBack 缓动函数（弹出效果）
        /// </summary>
        private float EaseOutBack(float t)
        {
            const float c1 = 1.70158f;
            const float c3 = c1 + 1f;
            return 1f + c3 * Mathf.Pow(t - 1f, 3f) + c1 * Mathf.Pow(t - 1f,
2f);
        }
        // ═══════════════════════════════════════════════
        // 位置计算
        // ═══════════════════════════════════════════════
        /// <summary>
```

```
        /// 计算安全生成位置
        /// </summary>
        private Vector3 CalculateSafePosition()
        {
            // 获取屏幕边界
            if (ScreenBoundaryManager.Instance == null)
            {
                GameLogger.LogWarning("[DrifterSpawnHelper]
ScreenBoundaryManager 不存在，使用默认位置");
                return new Vector3(0, 5f, 0);
            }
            float  left  =  ScreenBoundaryManager.Instance.ScreenLeft  +
config.minDistanceFromWall;
            float  right  =  ScreenBoundaryManager.Instance.ScreenRight  -
config.minDistanceFromWall;
            float  top  =  ScreenBoundaryManager.Instance.ScreenTop  -
config.minDistanceFromWall;
            float bottom = ScreenBoundaryManager.Instance.ScreenBottom +
config.minDistanceFromWall;
            // 只在上半区生成
            if (config.spawnInUpperHalf)
            {
                float midY = (top + bottom) * 0.5f;
                bottom = midY;
            }
            // 尝试找到安全位置（最多尝试 20 次）
            for (int i = 0; i < 20; i++)
            {
                float x = Random.Range(left, right);
                float y = Random.Range(bottom, top);
                Vector3 candidate = new Vector3(x, y, 0);
                if (ValidatePosition(candidate))
                {
                    return candidate;
                }
            }
            // 找不到理想位置，返回屏幕上方中央
            if (showDebugInfo)
            {
                GameLogger.LogWarning("[DrifterSpawnHelper] 无法找到理想安
全位置，使用备选位置");
            }
            return new Vector3(0, top - 1f, 0);
        }
        /// <summary>
        /// 验证位置是否安全
        /// </summary>
        private bool ValidatePosition(Vector3 position)
        {
            // 检查与塔的距离
```

```
            if (towerTransform != null)
            {
                float     distToTower     =     Vector3.Distance(position,
towerTransform.position);
                if (distToTower < config.minDistanceFromTower)
                {
                    return false;
                }
            }
            // 检查是否在安全区域内
            if (ScreenBoundaryManager.Instance != null)
            {
                float  left  =  ScreenBoundaryManager.Instance.ScreenLeft  +
config.minDistanceFromWall;
                float right = ScreenBoundaryManager.Instance.ScreenRight -
config.minDistanceFromWall;
                float  top  =  ScreenBoundaryManager.Instance.ScreenTop  -
config.minDistanceFromWall;
                float bottom = ScreenBoundaryManager.Instance.ScreenBottom
+ config.minDistanceFromWall;
                if (position.x < left || position.x > right ||
                    position.y < bottom || position.y > top)
                {
                    return false;
                }
            }
            return true;
        }
        // ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
        // 占位特效
        // ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
        /// <summary>
        /// 创建占位传送门特效（简单的缩放圆圈）
        /// </summary>
        private GameObject CreatePlaceholderPortalEffect(Vector3 position)
        {
            // 创建一个简单的圆形 Sprite 作为占位
            GameObject portal = new GameObject("PortalEffect_Placeholder");
            portal.transform.position = position;
            // 添加 SpriteRenderer
            SpriteRenderer sr = portal.AddComponent<SpriteRenderer>();
            sr.sprite = CreateCircleSprite();
            sr.color = new Color(0f, 1f, 1f, 0.8f); // 青色（霓虹风格）
            sr.sortingOrder = 100;
            // 启动缩放动画
            StartCoroutine(PortalEffectAnimation(portal));
            return portal;
        }
        /// <summary>
        /// 占位特效动画
```

```
        /// </summary>
        private IEnumerator PortalEffectAnimation(GameObject portal)
        {
            if (portal == null) yield break;
            Transform t = portal.transform;
            SpriteRenderer sr = portal.GetComponent<SpriteRenderer>();
            float duration = config.portalEffectDuration;
            float elapsed = 0f;
            while (elapsed < duration && portal != null)
            {
                elapsed += Time.deltaTime;
                float progress = elapsed / duration;
                // 缩放：0 → 1.5 → 0
                float scale;
                if (progress < 0.5f)
                {
                    scale = Mathf.Lerp(0f, 1.5f, progress * 2f);
                }
                else
                {
                    scale = Mathf.Lerp(1.5f, 0.5f, (progress - 0.5f) * 2f);
                }
                t.localScale = Vector3.one * scale;
                // 旋转
                t.Rotate(Vector3.forward, 360f * Time.deltaTime);
                // 透明度脉冲
                if (sr != null)
                {
                    float alpha = 0.5f + 0.3f * Mathf.Sin(progress * Mathf.PI
* 4f);
                    Color c = sr.color;
                    c.a = alpha;
                    sr.color = c;
                }
                yield return null;
            }
        }
        /// <summary>
        /// 创建简单的圆形 Sprite（运行时生成）
        /// </summary>
        private Sprite CreateCircleSprite()
        {
            int size = 64;
            Texture2D tex = new Texture2D(size, size, TextureFormat.RGBA32,
false);
            Color transparent = new Color(0, 0, 0, 0);
            Color white = Color.white;
            float center = size * 0.5f;
            float radius = size * 0.4f;
            float innerRadius = size * 0.3f;
```

```
            for (int y = 0; y < size; y++)
            {
                for (int x = 0; x < size; x++)
                {
                    float dist = Vector2.Distance(new Vector2(x, y), new
Vector2(center, center));
                    // 环形
                    if (dist < radius && dist > innerRadius)
                    {
                        float edge = 1f - Mathf.Abs(dist - (radius +
innerRadius) * 0.5f) / ((radius - innerRadius) * 0.5f);
                        tex.SetPixel(x, y, new Color(1, 1, 1, edge));
                    }
                    else
                    {
                        tex.SetPixel(x, y, transparent);
                    }
                }
            }
            tex.Apply();
            return Sprite.Create(tex, new Rect(0, 0, size, size), new
Vector2(0.5f, 0.5f), size);
        }
        // ──────────────────────────────────────────────
        // Gizmos
        // ──────────────────────────────────────────────
        private void OnDrawGizmos()
        {
            if (!showGizmos) return;
            if (ScreenBoundaryManager.Instance == null) return;
            // 绘制安全生成区域
            float left = ScreenBoundaryManager.Instance.ScreenLeft +
config.minDistanceFromWall;
            float right = ScreenBoundaryManager.Instance.ScreenRight -
config.minDistanceFromWall;
            float top = ScreenBoundaryManager.Instance.ScreenTop -
config.minDistanceFromWall;
            float bottom = ScreenBoundaryManager.Instance.ScreenBottom +
config.minDistanceFromWall;
            if (config.spawnInUpperHalf)
            {
                bottom = (top + bottom) * 0.5f;
            }
            Gizmos.color = new Color(0f, 1f, 0f, 0.3f);
            Vector3 center = new Vector3((left + right) * 0.5f, (top + bottom)
* 0.5f, 0);
            Vector3 size = new Vector3(right - left, top - bottom, 0.1f);
            Gizmos.DrawCube(center, size);
            Gizmos.color = Color.green;
            Gizmos.DrawWireCube(center, size);
```

```
            // 绘制塔的安全距离
            if (towerTransform != null)
            {
                Gizmos.color = new Color(1f, 0f, 0f, 0.3f);
                Gizmos.DrawWireSphere(towerTransform.position,
config.minDistanceFromTower);
            }
        }
    }
}namespace LightVsDecay.Core.Pool
{
    /// <summary>
    /// 对象池物体接口
    /// 所有需要使用对象池的物体都必须实现此接口
    /// </summary>
    public interface IPoolable
    {
        /// <summary>
        /// 从池中取出时调用（相当于 Awake/Start）
        /// </summary>
        void OnSpawn();
        /// <summary>
        /// 回收到池中时调用（相当于 OnDestroy）
        /// </summary>
        void OnDespawn();
        /// <summary>
        /// 获取该对象所属的池类型标识
        /// </summary>
        string PoolKey { get; }
    }
}// ============================================================
// EquipmentManager.cs
// 文件位置: Assets/Scripts/Logic/Equipment/EquipmentManager.cs
// 用途：装备系统核心管理器（堆叠背包版）
//
// 背包：同种同品质堆叠 → 合成消耗堆叠数
// 装备槽：独立记录 equipmentId + rarity + upgradeLevel
// 物品本身无等级，等级仅存在于装备槽
// ============================================================
using UnityEngine;
using LightVsDecay.Core;
using LightVsDecay.Data.Runtime;
using LightVsDecay.Data.SO;
namespace LightVsDecay.Logic.Equipment
{
    public class EquipmentManager : PersistentSingleton<EquipmentManager>
    {
        // ──────────────────────────────────────────────
        // Inspector
        // ──────────────────────────────────────────────
```

```
        [Header("配置")]
        [SerializeField] private EquipmentDatabase database;
        [Header("调试")]
        [SerializeField] private bool showDebugInfo = false;
#if UNITY_EDITOR
        [Header("Debug：初始背包（仅编辑器，格式 id:rarity:count）")]
        [SerializeField] private List<string> debugInitItems = new List<string>();
#endif
        // ─────────────────────────────────────────────
        // 事件
        // ─────────────────────────────────────────────
        /// <summary>背包内容变化（堆叠数增减/合成）</summary>
        public static event Action OnInventoryChanged;
        /// <summary>某槽位装备变化（装备/卸下/升级）</summary>
        public static event Action<EquipmentSlotType> OnEquipmentSlotChanged;
        /// <summary>图纸数量变化</summary>
        public static event Action<int> OnBlueprintsChanged;
        // ─────────────────────────────────────────────
        // 运行时数据
        // ─────────────────────────────────────────────
        private List<InventoryStack> _inventory = new List<InventoryStack>();
        private EquippedSlotData[] _slots = new EquippedSlotData[3]
        {
            EquippedSlotData.Empty(),
            EquippedSlotData.Empty(),
            EquippedSlotData.Empty(),
        };
        private int _blueprints = 0;
        // ─────────────────────────────────────────────
        // 公共只读属性
        // ─────────────────────────────────────────────
        public IReadOnlyList<InventoryStack> Inventory => _inventory;
        public int Blueprints => _blueprints;
        public EquipmentDatabase Database => database;
        /// <summary>获取指定槽位数据（空槽时 IsEmpty=true）</summary>
        public EquippedSlotData GetSlot(EquipmentSlotType slot) => _slots[(int)slot];
        /// <summary>获取指定槽位的 EquipmentData（空槽返回 null）</summary>
        public EquipmentData GetSlotData(EquipmentSlotType slot)
        {
            var s = _slots[(int)slot];
            return s.IsEmpty ? null : database?.GetById(s.equipmentId);
        }
        // ─────────────────────────────────────────────
        // 生命周期
        // ─────────────────────────────────────────────
        protected override void OnSingletonAwake()
```

```
        {
            if (database == null)
            {
                GameLogger.LogError("[EquipmentManager]        缺        少
EquipmentDatabase! ");
                return;
            }
            EquipmentDatabase.SetInstance(database);
            database.Initialize();
            LoadFromSave();
#if UNITY_EDITOR
            if (_inventory.Count == 0 && debugInitItems.Count > 0)
                ParseDebugItems();
#endif
        }
        // ═══════════════════════════════════════════════════
        // 背包操作
        // ═══════════════════════════════════════════════════
        /// <summary>向背包添加物品（自动堆叠）</summary>
        public void AddToInventory(string equipmentId, ItemRarity rarity,
int count = 1)
        {
            if (database?.GetById(equipmentId) == null)
            {
                GameLogger.LogWarning($"[EquipmentManager] 未 知 装 备 ID:
{equipmentId}");
                return;
            }
            var stack = FindStack(equipmentId, rarity);
            if (stack != null)
                stack.count += count;
            else
                _inventory.Add(new   InventoryStack(equipmentId,   rarity,
count));
            Save();
            OnInventoryChanged?.Invoke();
            if (showDebugInfo)
                GameLogger.Log($"[EquipManager]            获            得
{equipmentId}({rarity}) ×{count}");
        }
        /// <summary>获取指定种类的堆叠数量</summary>
        public int GetStackCount(string equipmentId, ItemRarity rarity)
            => FindStack(equipmentId, rarity)?.count ?? 0;
        /// <summary>是否存在任何可合成的组（≥3 个同种同品质）</summary>
        public bool HasAnyMergeable()
            =>  _inventory.Any(s  =>  s.count  >=  3  &&  s.rarity  <
ItemRarity.Legendary
                                    &&
database.GetMergeResult(database.GetById(s.equipmentId)) != null);
        // ═══════════════════════════════════════════════════
```

```
        // 装备/卸下
        // ═══════════════════════════════════════════════
        /// <summary>
        /// 装备背包中某个堆叠格到对应槽位
        /// 若槽位已有装备，先卸下（返回背包，等级清零）
        /// </summary>
        public EquipResult Equip(string equipmentId, ItemRarity rarity)
        {
            var data = database?.GetById(equipmentId);
            if (data == null) return EquipResult.DataNotFound;
            var stack = FindStack(equipmentId, rarity);
            if (stack == null || stack.count <= 0) return
EquipResult.NotEnoughItems;
            int slotIdx = (int)data.slotType;
            // 卸下已有装备（返回背包）
            if (!_slots[slotIdx].IsEmpty)
                UnequipInternal(data.slotType, returnToInventory: true);
            // 消耗 1 个
            stack.count--;
            if (stack.count <= 0) _inventory.Remove(stack);
            // 装备（初始 Lv.1）
            _slots[slotIdx] = EquippedSlotData.Create(equipmentId,
rarity);
            Save();
            OnInventoryChanged?.Invoke();
            OnEquipmentSlotChanged?.Invoke(data.slotType);
            if (showDebugInfo)
                GameLogger.Log($"[EquipManager] 装 备 :
{equipmentId}({rarity}) → 槽位{data.slotType}");
            return EquipResult.Success;
        }
        /// <summary>卸下槽位装备，返回背包</summary>
        public void Unequip(EquipmentSlotType slot)
            => UnequipInternal(slot, returnToInventory: true);
        private void UnequipInternal(EquipmentSlotType slot, bool
returnToInventory)
        {
            int idx = (int)slot;
            if (_slots[idx].IsEmpty) return;
            if (returnToInventory)
            {
                // 卸下后以基础物品（等级丢失）返回背包
                AddToInventoryNoSave(_slots[idx].equipmentId,
_slots[idx].rarity, 1);
            }
            _slots[idx] = EquippedSlotData.Empty();
            Save();
            OnEquipmentSlotChanged?.Invoke(slot);
            if (showDebugInfo)
                GameLogger.Log($"[EquipManager] 卸下槽位: {slot}");
```

```
        }
        // ═══════════════════════════════════════════
        // 升级槽位
        // ═══════════════════════════════════════════
        /// <summary>升级指定槽位装备等级（消耗金币+图纸）</summary>
        public LevelUpResult LevelUpSlot(EquipmentSlotType slot)
        {
            var slotData = _slots[(int)slot];
            if (slotData.IsEmpty)        return LevelUpResult.NotFound;
            var data = database?.GetById(slotData.equipmentId);
            if (data == null)            return LevelUpResult.NotFound;
            if    (slotData.upgradeLevel    >=    data.maxLevel)    return
LevelUpResult.AlreadyMaxLevel;
            int goldCost = data.GetLevelUpGoldCost(slotData.upgradeLevel);
            int             bpCost                                          =
data.GetLevelUpBlueprintCost(slotData.upgradeLevel);
            if     (ProgressManager.Instance     ==     null)     return
LevelUpResult.InsufficientResources;
            if  (ProgressManager.Instance.GoldCoins  <  goldCost)  return
LevelUpResult.InsufficientGold;
            if  (_blueprints  <  bpCost)                            return
LevelUpResult.InsufficientBlueprints;
            ProgressManager.Instance.ConsumeGoldCoins(goldCost);
            _blueprints -= bpCost;
            slotData.upgradeLevel++;
            Save();
            OnEquipmentSlotChanged?.Invoke(slot);
            if (showDebugInfo)
                GameLogger.Log($"[EquipManager]    {slot}    升    级      →
Lv.{slotData.upgradeLevel}（消耗 {goldCost}金 {bpCost}图纸）");
            return LevelUpResult.Success;
        }
        /// <summary>无损重置：退还所有升级消耗，等级归 1</summary>
        public void ResetSlotLevel(EquipmentSlotType slot)
        {
            var slotData = _slots[(int)slot];
            if (slotData.IsEmpty || slotData.upgradeLevel <= 1) return;
            var data = database?.GetById(slotData.equipmentId);
            if (data == null) return;
            // 退还从 Lv.1 到当前等级的全部消耗
            int refundGold = 0, refundBp = 0;
            for (int lv = 1; lv < slotData.upgradeLevel; lv++)
            {
                refundGold += data.GetLevelUpGoldCost(lv);
                refundBp   += data.GetLevelUpBlueprintCost(lv);
            }
            ProgressManager.Instance?.AddGoldCoins(refundGold);
            _blueprints += refundBp;
            slotData.upgradeLevel = 1;
            Save();
```

```
            OnEquipmentSlotChanged?.Invoke(slot);
            OnBlueprintsChanged?.Invoke(_blueprints);
            if (showDebugInfo)
                GameLogger.Log($"[EquipManager] {slot} 无损重置，退还
{refundGold}金 {refundBp}图纸");
        }
        // ═══════════════════════════════════════════════
        // 合成
        // ═══════════════════════════════════════════════
        /// <summary>对指定堆叠格合成一次（消耗 3 个 → 获得 1 个更高品质）
</summary>
        public bool Merge(string equipmentId, ItemRarity rarity)
        {
            var stack = FindStack(equipmentId, rarity);
            if (stack == null || stack.count < 3) return false;
            var sourceData = database?.GetById(equipmentId);
            if (sourceData == null) return false;
            var resultData = database.GetMergeResult(sourceData);
            if (resultData == null) return false;
            stack.count -= 3;
            if (stack.count <= 0) _inventory.Remove(stack);
            AddToInventoryNoSave(resultData.equipmentId,
resultData.rarity, 1);
            Save();
            OnInventoryChanged?.Invoke();
            if (showDebugInfo)
                GameLogger.Log($"[EquipManager] 合成：
3×{equipmentId}({rarity}) → 
{resultData.equipmentId}({resultData.rarity})");
            return true;
        }
        /// <summary>一键合成：自动合成所有可合成的组，返回合成次数</summary>
        public int AutoMergeAll()
        {
            int total = 0;
            bool merged = true;
            while (merged)
            {
                merged = false;
                foreach (var s in _inventory.ToList())
                {
                    if (s.count >= 3 && s.rarity < ItemRarity.Legendary)
                    {
                        if (Merge(s.equipmentId, s.rarity))
                        { total++; merged = true; break; }
                    }
                }
            }
            if (showDebugInfo && total > 0)
```

```
                GameLogger.Log($"[EquipManager] 一键合成完成，共 {total} 次
");
            return total;
        }
        /// <summary>一键装备：每个槽位自动装备背包中最高品质的物品</summary>
        public void AutoEquipBest()
        {
            foreach          (EquipmentSlotType          slot          in
Enum.GetValues(typeof(EquipmentSlotType)))
            {
                // 找对应槽位中最高品质的背包物品
                var slotId = GetSlotEquipmentId(slot);
                var best   = _inventory
                    .Where(s       =>       s.count       >       0       &&
database?.GetById(s.equipmentId)?.slotType == slot)
                    .OrderByDescending(s => (int)s.rarity)
                    .FirstOrDefault();
                if (best == null) continue;
                // 如果背包最好的和已装备的一样，跳过
                var current = _slots[(int)slot];
                if    (!current.IsEmpty    &&    current.equipmentId    ==
best.equipmentId
                    && current.rarity >= best.rarity) continue;
                Equip(best.equipmentId, best.rarity);
            }
        }
        // ═══════════════════════════════════════════════
        // 分解
        // ═══════════════════════════════════════════════
        /// <summary>分解背包中指定堆叠格的 1 个物品，获得图纸</summary>
        public int Decompose(string equipmentId, ItemRarity rarity)
        {
            var stack = FindStack(equipmentId, rarity);
            if (stack == null || stack.count <= 0) return 0;
            int gain = (int)rarity + 1;   // 绿+2, 蓝+3, 紫+4, 橙+5
            stack.count--;
            if (stack.count <= 0) _inventory.Remove(stack);
            _blueprints += gain;
            Save();
            OnInventoryChanged?.Invoke();
            OnBlueprintsChanged?.Invoke(_blueprints);
            if (showDebugInfo)
                GameLogger.Log($"[EquipManager]            分            解
{equipmentId}({rarity})，获得 {gain} 图纸");
            return gain;
        }
        // ═══════════════════════════════════════════════
        // 合计属性 & 塔外观档次
        // ═══════════════════════════════════════════════
```

```
        /// <summary>计算三件装备的合计属性（供 TurretHealth 等读取）
</summary>
        public EquipmentStats GetTotalStats()
        {
            var total = new EquipmentStats();
            foreach (EquipmentSlotType slot in
Enum.GetValues(typeof(EquipmentSlotType)))
            {
                var s    = _slots[(int)slot];
                var data = s.IsEmpty ? null :
database?.GetById(s.equipmentId);
                if (data == null) continue;
                total = total + data.GetStatsAtLevel(s.upgradeLevel);
            }
            return total;
        }
        /// <summary>
        /// 塔外观档次（0=灰/无装备，1=绿，2=蓝，3=紫，4=橙）
        /// 规则：任一槽位为空→0；否则取三槽最低品质对应档次
        /// </summary>
        public int GetOverallTowerTier()
        {
            foreach (var s in _slots)
                if (s.IsEmpty) return 0;
            int lowest = (int)ItemRarity.Legendary;
            foreach (var s in _slots)
                lowest = Mathf.Min(lowest, (int)s.rarity);
            // ItemRarity: Common=0(不存在实际装备) Uncommon=1 Rare=2 Epic=3
Legendary=4
            return Mathf.Clamp(lowest, 0, 4);
        }
        /// <summary>直接增加图纸数量（结算奖励用）</summary>
        public void AddBlueprints(int count)
        {
            if (count <= 0) return;
            _blueprints += count;
            Save();
            OnBlueprintsChanged?.Invoke(_blueprints);
            if (showDebugInfo)
                GameLogger.Log($"[EquipManager] 获得图纸 ×{count}，当前共
{_blueprints}");
        }
        // ──────────────────────────────────────────
        // 辅助
        // ──────────────────────────────────────────
        private InventoryStack FindStack(string id, ItemRarity r)
            => _inventory.FirstOrDefault(s => s.equipmentId == id &&
s.rarity == r);
        private string GetSlotEquipmentId(EquipmentSlotType slot)
            => _slots[(int)slot].equipmentId;
```

```
        private void AddToInventoryNoSave(string id, ItemRarity r, int
count)
        {
            var s = FindStack(id, r);
            if (s != null) s.count += count;
            else _inventory.Add(new InventoryStack(id, r, count));
        }
        // ──────────────────────────────────────────────
        // 存档
        // ──────────────────────────────────────────────
        private void Save()
        {
            var d = new EquipmentSaveData
            {
                inventory  = _inventory,
                slots      = _slots,
                blueprints = _blueprints,
            };
            d.Save();
        }
        private void LoadFromSave()
        {
            var d = EquipmentSaveData.Load();
            _inventory  = d.inventory ?? new List<InventoryStack>();
            _slots      = d.slots     ?? new EquippedSlotData[3]
                {   EquippedSlotData.Empty(),   EquippedSlotData.Empty(),
EquippedSlotData.Empty() };
            _blueprints = d.blueprints;
            if (showDebugInfo)
                GameLogger.Log($"[EquipManager]  读 档 完 成 ： 背 包
{_inventory.Count} 格，图纸 {_blueprints}");
        }
        /// <summary>
        /// 重置所有装备数据（内存 + PlayerPrefs），供 PlayerDataResetTool 调
用
        /// </summary>
        public void ResetAll()
        {
            _inventory  = new List<InventoryStack>();
            _slots      = new EquippedSlotData[3]
            {
                EquippedSlotData.Empty(),
                EquippedSlotData.Empty(),
                EquippedSlotData.Empty(),
            };
            _blueprints = 0;
            EquipmentSaveData.Reset();   // 同时清 PlayerPrefs
            OnInventoryChanged?.Invoke();
            OnBlueprintsChanged?.Invoke(0);
            if (showDebugInfo)
```

```
                GameLogger.Log("[EquipmentManager] ResetAll: 内存+存档全部
清除");
        }
#if UNITY_EDITOR
        private void ParseDebugItems()
        {
            foreach (var entry in debugInitItems)
            {
                //   格   式   :   equipmentId:rarityInt:count      例   如
core_uncommon:1:5
                var parts = entry.Split(':');
                if (parts.Length < 2) continue;
                if (!int.TryParse(parts[1], out int r)) continue;
                int cnt = parts.Length >= 3 && int.TryParse(parts[2], out int
c) ? c : 1;
                AddToInventory(parts[0], (ItemRarity)r, cnt);
            }
        }
#endif
    }
    // ═══════════════════════════════════════════════════
    // 结果枚举
    // ═══════════════════════════════════════════════════
    public enum EquipResult
    {
        Success, DataNotFound, NotEnoughItems
    }
    public enum LevelUpResult
    {
        Success, NotFound, AlreadyMaxLevel,
        InsufficientResources, InsufficientGold, InsufficientBlueprints
    }
}// ============================================================
// ChapterConfig.cs
// 文件位置: Assets/Scripts/Data/SO/ChapterConfig.cs
// 用途：单章节配置（ScriptableObject）
// ============================================================
using UnityEngine;
using LightVsDecay.Core;
namespace LightVsDecay.Data.SO
{
    // ═══════════════════════════════════════════════════
    // 章节特殊机制枚举
    // ═══════════════════════════════════════════════════
    /// <summary>
    /// 章节特殊机制类型
    /// </summary>
    public enum ChapterMechanicType
    {
        /// <summary>无特殊机制（章节 1：深暗虚空）</summary>
```

```
        None = 0,
        /// <summary>岩浆机制（章节 2：熔岩虚空）- 坦克死后残留岩浆斑，Boss 喷火球</summary>
        Lava = 1,
        /// <summary>冰封机制（章节 3：极寒虚空）- 怪物带冰甲，Boss 可冰封炮塔</summary>
        Frozen = 2
    }
    // ═══════════════════════════════════════════════
    // 难度倍率配置（内嵌类）
    // ═══════════════════════════════════════════════
    /// <summary>
    /// 难度倍率设置
    /// 用于调整同一章节不同难度下的数值
    /// </summary>
    [System.Serializable]
    public class DifficultySettings
    {
        [Tooltip("难度等级 (1-5)")]
        public int difficultyLevel = 1;
        [Tooltip("难度显示名称")]
        public string displayName = "普通";
        [Header("敌人倍率")]
        [Tooltip("敌人生命值倍率")]
        [Range(1f, 5f)]
        public float enemyHealthMultiplier = 1f;
        [Tooltip("敌人数量倍率")]
        [Range(1f, 3f)]
        public float enemyCountMultiplier = 1f;
        [Tooltip("敌人移动速度倍率")]
        [Range(1f, 2f)]
        public float enemySpeedMultiplier = 1f;
        [Header("Boss 倍率")]
        [Tooltip("Boss 生命值倍率")]
        [Range(1f, 5f)]
        public float bossHealthMultiplier = 1f;
        [Tooltip("Boss 攻击力倍率")]
        [Range(1f, 3f)]
        public float bossAttackMultiplier = 1f;
        [Header("奖励倍率")]
        [Tooltip("金币掉落倍率")]
        [Range(1f, 3f)]
        public float coinDropMultiplier = 1f;
        [Tooltip("经验掉落倍率")]
        [Range(1f, 2f)]
        public float expDropMultiplier = 1f;
        /// <summary>
        /// 创建默认难度配置
        /// </summary>
        public static DifficultySettings CreateDefault(int level)
```

```
        {
            return new DifficultySettings
            {
                difficultyLevel = level,
                displayName = GetDefaultName(level),
                enemyHealthMultiplier = 1f + (level - 1) * 0.3f,      // 1.0,
1.3, 1.6, 1.9, 2.2
                enemyCountMultiplier = 1f + (level - 1) * 0.15f,      // 1.0,
1.15, 1.3, 1.45, 1.6
                enemySpeedMultiplier = 1f + (level - 1) * 0.05f,      // 1.0,
1.05, 1.1, 1.15, 1.2
                bossHealthMultiplier = 1f + (level - 1) * 0.4f,       // 1.0,
1.4, 1.8, 2.2, 2.6
                bossAttackMultiplier = 1f + (level - 1) * 0.2f,       // 1.0,
1.2, 1.4, 1.6, 1.8
                coinDropMultiplier = 1f + (level - 1) * 0.25f,        // 1.0,
1.25, 1.5, 1.75, 2.0
                expDropMultiplier = 1f + (level - 1) * 0.1f           // 1.0,
1.1, 1.2, 1.3, 1.4
            };
        }
        private static string GetDefaultName(int level)
        {
            return level switch
            {
                1 => "普通",
                2 => "困难",
                3 => "噩梦",
                4 => "地狱",
                5 => "深渊",
                _ => $"难度{level}"
            };
        }
    }
    // ═══════════════════════════════════════════════════
    // 章节配置 ScriptableObject
    // ═══════════════════════════════════════════════════
    /// <summary>
    /// 章节配置 (ScriptableObject)
    /// 定义单个章节的所有配置数据
    /// </summary>
    [CreateAssetMenu(fileName     =     "Chapter_New",     menuName     =
"LightVsDecay/Chapter Config", order = 10)]
    public class ChapterConfig : ScriptableObject
    {
        // ═══════════════════════════════════════════════
        // 基本信息
        // ═══════════════════════════════════════════════
        [Header("═══ 基本信息 ═══")]
        [Tooltip("章节索引 (0-based，用于程序逻辑)")]
```

```
        public int chapterIndex = 0;
        [Tooltip("章节显示编号 (1-based，用于 UI 显示)")]
        public int chapterNumber = 1;
        [Tooltip("章节名称（中文）")]
        public string chapterName = "新章节";
        [Tooltip("章节描述")]
        [TextArea(2, 4)]
        public string chapterDescription = "";
        // ═══════════════════════════════════════════════════
        // 视觉资源
        // ═══════════════════════════════════════════════════
        [Header("═══ 视觉资源 ═══")]
        [Tooltip("章节选择界面的背景图")]
        public Sprite chapterCardImage;
        [Tooltip("战斗场景背景图")]
        public Sprite battleBackgroundImage;
        [Tooltip("章节主题色（用于 UI 高亮等）")]
        public Color themeColor = new Color(0f, 1f, 1f, 1f); // 默认青色
        [Tooltip("流体怪物底色（MetaballsThreshold shader 的 _Color）")]
        public Color enemyBlobColor = new Color(0.1f, 0f, 0.2f, 1f); // 默
认深紫/黑（第 1 章
        // ═══════════════════════════════════════════════════
        // 音频资源
        // ═══════════════════════════════════════════════════
        [Header("═══ 音频资源 ═══")]
        [Tooltip("章节战斗 BGM（不设置则使用默认 BGM）")]
        public AudioClip battleBGM;
        [Tooltip("Boss 战 BGM（不设置则继续播放战斗 BGM）")]
        public AudioClip bossBGM;
        // ═══════════════════════════════════════════════════
        // 游戏配置
        // ═══════════════════════════════════════════════════
        [Header("═══ 游戏配置 ═══")]
        [Tooltip("波次配置（定义敌人生成规则）")]
        public WaveConfig waveConfig;
        [Tooltip("Boss 预制体（覆盖 WaveConfig 中的默认 Boss）")]
        public GameObject bossPrefab;
        // ═══════════════════════════════════════════════════
        // 特殊机制
        // ═══════════════════════════════════════════════════
        [Header("═══ 特殊机制 ═══")]
        [Tooltip("章节特殊机制类型")]
        public         ChapterMechanicType         mechanicType         =
ChapterMechanicType.None;
        // ═══════════════════════════════════════════════════
        // 难度设置
        // ═══════════════════════════════════════════════════
        [Header("═══ 难度设置 ═══")]
        [Tooltip("5 个难度等级的配置")]
```

```
        public DifficultySettings[] difficulties = new DifficultySettings[5];
        // ═══════════════════════════════════════════════════
        // 常量
        // ═══════════════════════════════════════════════════
        public const int MAX_DIFFICULTY = 5;
        // ═══════════════════════════════════════════════════
        // 公共方法
        // ═══════════════════════════════════════════════════
        /// <summary>
        /// 获取指定难度的配置
        /// </summary>
        /// <param name="difficulty">难度等级 (1-5)</param>
        /// <returns>难度配置，如果不存在返回 null</returns>
        public DifficultySettings GetDifficulty(int difficulty)
        {
            int index = difficulty - 1;
            if (index >= 0 && index < difficulties.Length)
            {
                return difficulties[index];
            }
            GameLogger.LogWarning($"[ChapterConfig] 难度 {difficulty} 不存在！");
            return null;
        }
        /// <summary>
        /// 获取章节显示标题（带编号）
        /// </summary>
        public string DisplayTitle => $"{chapterNumber}.{chapterName}";
        /// <summary>
        /// 检查是否有特殊机制
        /// </summary>
        public bool HasSpecialMechanic => mechanicType != ChapterMechanicType.None;
        // ═══════════════════════════════════════════════════
        // 编辑器工具
        // ═══════════════════════════════════════════════════
#if UNITY_EDITOR
        /// <summary>
        /// 生成默认 5 个难度配置
        /// </summary>
        [ContextMenu("生成默认难度配置")]
        public void GenerateDefaultDifficulties()
        {
            difficulties = new DifficultySettings[MAX_DIFFICULTY];
            for (int i = 0; i < MAX_DIFFICULTY; i++)
            {
                difficulties[i] = DifficultySettings.CreateDefault(i + 1);
            }
            UnityEditor.EditorUtility.SetDirty(this);
```

```
            GameLogger.Log($"[ChapterConfig] 已生成 {MAX_DIFFICULTY} 个默
认难度配置");
        }
        /// <summary>
        /// 验证配置完整性
        /// </summary>
        [ContextMenu("验证配置")]
        public void ValidateConfig()
        {
            bool isValid = true;
            if (string.IsNullOrEmpty(chapterName))
            {
                GameLogger.LogWarning($"[ChapterConfig] 章节名称为空！");
                isValid = false;
            }
            if (chapterCardImage == null)
            {
                GameLogger.LogWarning($"[ChapterConfig] 章节卡片图片未设置！
");
                isValid = false;
            }
            if (waveConfig == null)
            {
                GameLogger.LogWarning($"[ChapterConfig] 波次配置未设置！");
                isValid = false;
            }
            if (difficulties == null || difficulties.Length !=
MAX_DIFFICULTY)
            {
                GameLogger.LogWarning($"[ChapterConfig] 难度配置数量不正确！
应为 {MAX_DIFFICULTY} 个");
                isValid = false;
            }
            if (isValid)
            {
                GameLogger.Log($"[ChapterConfig] 配置验证通过 ✓");
            }
        }
#endif
    }
}// ============================================================
// SettingsPanel.cs
// 文件位置: Assets/Scripts/UI/Panels/SettingsPanel.cs
// 用途：设置/暂停面板控制器
// 功能：音乐/音效开关、返回主页、重新开始
// ============================================================
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.EventSystems;
using LightVsDecay.Audio;
```

```
using LightVsDecay.Logic;
using LightVsDecay.Core;
namespace LightVsDecay.UI
{
    /// <summary>
    /// 设置面板控制器
    /// 主界面：仅显示音乐/音效开关
    /// 战斗场景：显示音乐/音效开关 + 返回主页/重新开始按钮
    /// </summary>
    public class SettingsPanel : MonoBehaviour
    {
        // ═══════════════════════════════════════════════
        // UI 组件引用 - 音频开关
        // ═══════════════════════════════════════════════
        [Header("═══ 音频开关 ═══")]
        [Tooltip("音乐开关 Toggle")]
        [SerializeField] private Toggle musicToggle;
        [Tooltip("音效开关 Toggle")]
        [SerializeField] private Toggle soundToggle;
        [Header("═══ 开关滑块（Checkmark）═══")]
        [Tooltip("音乐开关滑块")]
        [SerializeField] private RectTransform musicCheckmark;
        [Tooltip("音效开关滑块")]
        [SerializeField] private RectTransform soundCheckmark;
        [Header("═══ 滑块位置配置 ═══")]
        [Tooltip("开启时的 X 位置")]
        [SerializeField] private float checkmarkOnPosX = 140f;
        [Tooltip("关闭时的 X 位置")]
        [SerializeField] private float checkmarkOffPosX = -140f;
        [Header("═══ 开关填充图（可选，用于视觉反馈）═══")]
        [Tooltip("音乐开关填充图（开启时显示）")]
        [SerializeField] private GameObject musicFillImage;
        [Tooltip("音效开关填充图（开启时显示）")]
        [SerializeField] private GameObject soundFillImage;
        // ═══════════════════════════════════════════════
        // UI 组件引用 - 底部按钮区域（战斗场景显示）
        // ═══════════════════════════════════════════════
        [Header("═══ 底部按钮区域（BottomArea）═══")]
        [Tooltip("底部按钮区域根节点")]
        [SerializeField] private GameObject bottomArea;
        [Tooltip("返回主页按钮")]
        [SerializeField] private Button homeButton;
        [Tooltip("重新开始按钮")]
        [SerializeField] private Button restartButton;
        [Header("═══ 按钮禁用颜色 ═══")]
        [Tooltip("按钮禁用时的颜色")]
        [SerializeField] private Color disabledButtonColor = new Color(0.5f,
0.5f, 0.5f, 1f);
        // ═══════════════════════════════════════════════
        // UI 组件引用 - 内容区域（用于点击空白关闭判断）
```

```
// ═══════════════════════════════════════════════════
[Header("═══ 关闭按钮 ═══")]
[Tooltip("背景遮罩按钮（点击空白处关闭）")]
[SerializeField] private Button backgroundButton;
[Tooltip("右上角 X 关闭按钮（可选）")]
[SerializeField] private Button closeButton;
[Header("═══ 调试 ═══")]
[SerializeField] private bool showDebugInfo = false;
// ═══════════════════════════════════════════════════
// 运行时状态
// ═══════════════════════════════════════════════════
private bool showBottomArea = false;  // 是否显示底部按钮区域
private bool isInBattleScene = false; // 是否在战斗场景
private Image restartButtonImage;
private Color restartButtonOriginalColor;
// ═══════════════════════════════════════════════════
// Unity 生命周期
// ═══════════════════════════════════════════════════
private void Awake()
{
    // 缓存重新开始按钮的图片组件
    if (restartButton != null)
    {
        restartButtonImage = restartButton.GetComponent<Image>();
        if (restartButtonImage != null)
        {
            restartButtonOriginalColor = restartButtonImage.color;
        }
    }
}
private void Start()
{
    SetupToggles();
    SetupButtons();
}
private void OnEnable()
{
    // 每次显示时刷新 Toggle 状态
    RefreshToggleStates();
    // 如果在战斗场景，暂停游戏
    if (isInBattleScene)
    {
        if (GameManager.Instance != null)
        {
            GameManager.Instance.PauseGame();
        }
        // 更新重新开始按钮状态
        UpdateRestartButtonState();
    }
    if (showDebugInfo)
```

```
            {
                GameLogger.Log($"[SettingsPanel] OnEnable - 战斗场景: 
{isInBattleScene}, 显示底部: {showBottomArea}");
            }
        }
        // ═══════════════════════════════════════════════
        // 初始化
        // ═══════════════════════════════════════════════
        private void SetupToggles()
        {
            if (musicToggle != null)
            {

musicToggle.onValueChanged.AddListener(OnMusicToggleChanged);
            }
            if (soundToggle != null)
            {

soundToggle.onValueChanged.AddListener(OnSoundToggleChanged);
            }
            RefreshToggleStates();
        }
        private void SetupButtons()
        {
            if    (homeButton                                  !=    null)
homeButton.onClick.AddListener(OnHomeButtonClicked);
            if      (restartButton                           !=      null)
restartButton.onClick.AddListener(OnRestartButtonClicked);
            // ★ 新增
            if           (backgroundButton           !=           null)
backgroundButton.onClick.AddListener(OnCloseClicked);
            if    (closeButton                                 !=    null)
closeButton.onClick.AddListener(OnCloseClicked);
        }
        private void RefreshToggleStates()
        {
            if (AudioManager.Instance == null) return;
            // 同步 Toggle 状态
            if (musicToggle != null)
            {
                bool musicOn = AudioManager.Instance.BGMEnabled;
                musicToggle.SetIsOnWithoutNotify(musicOn);
                UpdateCheckmarkPosition(musicCheckmark, musicOn);
                UpdateFillImage(musicFillImage, musicOn);
            }
            if (soundToggle != null)
            {
                bool soundOn = AudioManager.Instance.SFXEnabled;
                soundToggle.SetIsOnWithoutNotify(soundOn);
                UpdateCheckmarkPosition(soundCheckmark, soundOn);
```

```
            UpdateFillImage(soundFillImage, soundOn);
        }
    }
    // ═══════════════════════════════════════════════
    // Toggle 回调
    // ═══════════════════════════════════════════════
    private void OnMusicToggleChanged(bool isOn)
    {
        if (AudioManager.Instance != null)
        {
            AudioManager.Instance.BGMEnabled = isOn;
        }
        // 更新 Checkmark 位置
        UpdateCheckmarkPosition(musicCheckmark, isOn);
        UpdateFillImage(musicFillImage, isOn);
        // 播放按钮音效
        PlayButtonSound();
        if (showDebugInfo)
        {
            GameLogger.Log($"[SettingsPanel] 音乐开关: {isOn}");
        }
    }
    private void OnSoundToggleChanged(bool isOn)
    {
        if (AudioManager.Instance != null)
        {
            AudioManager.Instance.SFXEnabled = isOn;
        }
        // 更新 Checkmark 位置
        UpdateCheckmarkPosition(soundCheckmark, isOn);
        UpdateFillImage(soundFillImage, isOn);
        PlayButtonSound();
        if (showDebugInfo)
        {
            GameLogger.Log($"[SettingsPanel] 音效开关: {isOn}");
        }
    }
    /// <summary>
    /// 更新 Checkmark 滑块位置
    /// </summary>
    private void UpdateCheckmarkPosition(RectTransform checkmark, bool
isOn)
    {
        if (checkmark == null) return;
        Vector2 pos = checkmark.anchoredPosition;
        pos.x = isOn ? checkmarkOnPosX : checkmarkOffPosX;
        checkmark.anchoredPosition = pos;
    }
    private void UpdateFillImage(GameObject fillImage, bool isOn)
    {
```

```
            if (fillImage != null)
            {
                fillImage.SetActive(isOn);
            }
        }
        // ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
        // 按钮回调
        // ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
        private void OnHomeButtonClicked()
        {
            PlayButtonSound();
            if (showDebugInfo)
            {
                GameLogger.Log("[SettingsPanel] 点击返回主页");
            }
            // 隐藏面板
            Hide();
            // 返回主菜单
            if (GameManager.Instance != null)
            {
                GameManager.Instance.LoadMainMenu();
            }
        }
        private void OnRestartButtonClicked()
        {
            // 检查能量是否足够
            int currentEnergy = 0;
            if (ProgressManager.Instance != null)
            {
                currentEnergy = ProgressManager.Instance.Energy;
            }
            else
            {
                currentEnergy = PlayerPrefs.GetInt("PlayerEnergy", 5);
            }
            if (currentEnergy <= 0)
            {
                if (showDebugInfo)
                {
                    GameLogger.Log("[SettingsPanel] 能量不足，无法重新开始
");
                }
                return;
            }
            PlayButtonSound();
            if (showDebugInfo)
            {
                GameLogger.Log("[SettingsPanel] 点击重新开始");
            }
            // 扣除能量
```

```
            if (ProgressManager.Instance != null)
            {
                ProgressManager.Instance.ConsumeEnergy(1);
            }
            else
            {
                PlayerPrefs.SetInt("PlayerEnergy", currentEnergy - 1);
                PlayerPrefs.Save();
            }
            // 隐藏面板
            Hide();
            // 重新开始游戏
            if (GameManager.Instance != null)
            {
                GameManager.Instance.RestartGame();
            }
        }
        /// <summary>
        /// 更新重新开始按钮状态（能量不足时置灰）
        /// </summary>
        private void UpdateRestartButtonState()
        {
            if (restartButton == null) return;
            int currentEnergy = 0;
            if (ProgressManager.Instance != null)
            {
                currentEnergy = ProgressManager.Instance.Energy;
            }
            else
            {
                currentEnergy = PlayerPrefs.GetInt("PlayerEnergy", 5);
            }
            bool hasEnergy = currentEnergy > 0;
            restartButton.interactable = hasEnergy;
            if (restartButtonImage != null)
            {
                restartButtonImage.color       =       hasEnergy       ?
restartButtonOriginalColor : disabledButtonColor;
            }
            if (showDebugInfo)
            {
                GameLogger.Log($"[SettingsPanel] 重新开始按钮状态：能量
={currentEnergy}, 可用={hasEnergy}");
            }
        }
        // ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
        // 关闭界面
        // ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
        private void OnCloseClicked()
        {
```

```
            PlayButtonSound();
            Hide();
            if (showDebugInfo)
                GameLogger.Log("[SettingsPanel] 点击关闭");
        }
        // ═══════════════════════════════════════════════
        // 公共方法
        // ═══════════════════════════════════════════════
        /// <summary>
        /// 显示设置面板
        /// </summary>
        /// <param name="showBottom">是否显示底部按钮区域（战斗场景为 true）
</param>
        public void Show(bool showBottom = false)
        {
            showBottomArea = showBottom;
            isInBattleScene = showBottom;
            // 设置底部区域显隐
            if (bottomArea != null)
            {
                bottomArea.SetActive(showBottom);
            }
            // 显示面板
            gameObject.SetActive(true);
            // ★ 清除 EventSystem 当前选中，防止首次点击被吞

UnityEngine.EventSystems.EventSystem.current?.SetSelectedGameObject(nul
l);
            if (showDebugInfo)
            {
                GameLogger.Log($"[SettingsPanel] Show - 显示底部区域：
{showBottom}");
            }
        }
        /// <summary>
        /// 隐藏设置面板
        /// </summary>
        public void Hide()
        {
            gameObject.SetActive(false);
            // 如果在战斗场景，恢复游戏
            if (isInBattleScene)
            {
                if (GameManager.Instance != null)
                {
                    GameManager.Instance.ResumeGame();
                }
                // 重启激光音效
                if (AudioManager.Instance != null)
                {
```

```
                    AudioManager.Instance.StartLaserLoop();
                }
            }
            if (showDebugInfo)
            {
                GameLogger.Log("[SettingsPanel] Hide");
            }
        }
        // ═══════════════════════════════════════════════════
        // 辅助方法
        // ═══════════════════════════════════════════════════
        private void PlayButtonSound()
        {
            if (AudioManager.Instance != null)
            {
                AudioManager.Instance.PlayButtonClick();
            }
        }
    }
}// ============================================================
// BossController.cs
// 文件位置: Assets/Scripts/Logic/Boss/BossController.cs
// 用途：Boss 行为状态机主控制器 (The Corruptor - 污染之核)
// 【重构】分离 Charge(快招) 和 Press(慢招) 两个主动技能
// 【新增】冰冻状态 + 霸体系统 + 控制递减机制
// ============================================================
using UnityEngine;
using LightVsDecay.Audio;
using LightVsDecay.Core;
using LightVsDecay.Core.Pool;
using LightVsDecay.Data;
using LightVsDecay.Data.SO;
using LightVsDecay.Logic.Enemy;
using LightVsDecay.Logic.Player;
using LightVsDecay.Logic.Statistics;
using LightVsDecay.UI.FloatingText;
#if DOTWEEN
using DG.Tweening;
#endif
namespace LightVsDecay.Logic.Boss
{
    /// <summary>
    /// Boss 状态飘字类型
    /// </summary>
    public enum BossStatusTextType
    {
        Unstoppable,    // 霸体
        Frozen,         // 冰冻
        Enraged,        // 狂暴
        Interrupted,    // 被打断
```

```
        Countered,      // 被反击
        Overload,       // 过载
        Exhausted       // 疲劳
    }
    /// <summary>
    /// Boss 行为状态
    /// </summary>
    public enum BossState
    {
        Spawn,      // 入场
        Idle,       // 待机/游走
        Summon,     // 召唤爪牙（被动技能，可打断 Idle）
        Charge,     // 野蛮冲撞（主动技能 A - 快招/反应测试）
        Press,      // 重力碾压（主动技能 B - 慢招/物理角力）
        Stun,       // 僵直/虚弱
        Frozen      // 【新增】冰冻状态
    }
    /// <summary>
    /// Boss 控制器 - 污染之核 (The Corruptor)
    /// 状态机循环：Spawn -> Idle -> (Summon 可打断) -> Charge/Press(50/50)
-> Stun -> Idle ...
    /// 【新增】支持冰冻状态和霸体机制
    /// </summary>
    [RequireComponent(typeof(Rigidbody2D))]
    [RequireComponent(typeof(BossHealth))]
    public class BossController : MonoBehaviour
    {
        // ═══════════════════════════════════════════════
        // Inspector 配置
        // ═══════════════════════════════════════════════
        [Header("配置")]
        [Tooltip("Boss 行为配置")]
        [SerializeField] private BossConfig config;
        [Header("组件引用")]
        [Tooltip("眼睛控制器")]
        [SerializeField] private BossEyeController eyeController;
        [Tooltip("身体 Transform（用于震动效果）")]
        [SerializeField] private Transform bodyTransform;
        [Tooltip("所有身体渲染器（用于颜色效果）")]
        [SerializeField] private SpriteRenderer[] bodyRenderers;
        [Tooltip("红色身体特效（怒目时显示）")]
        [SerializeField] private GameObject redBodyEffect;
        [Header("调试")]
        [SerializeField] private bool showDebugInfo = false;
        // ═══════════════════════════════════════════════
        // 运行时状态
        // ═══════════════════════════════════════════════
        private BossState currentState = BossState.Spawn;
        private BossState stateBeforeFrozen = BossState.Idle; // 【新增】
冰冻前的状态
```

```
        private Rigidbody2D rb;
        private BossHealth bossHealth;
        // 位置缓存
        private Vector3 spawnPosition;
        private Vector3 battleAnchorPosition;
        private float screenMinX, screenMaxX;
        // 颜色缓存
        private Color[] originalColors;
        // Idle 状态
        private float idleTimer = 0f;
        private float currentIdleDuration;
        private float idleMoveTargetX;
        // Summon 冷却
        private float summonCooldownTimer = 0f;
        private bool summonCooldownReady = false;
        // Pollution 计时
        private float pollutionTimer = 0f;
        // Charge 状态
        private bool chargeInterrupted = false;
        private int chargeHitCount = 0;
        // Press 状态
        private bool isPressing = false;
        private bool isPressPhase3Active = false;
        private float pressDownForce = 0f;
        private Vector2 accumulatedPushForce = Vector2.zero;
        private bool isBeingPushed = false;
        // Press 角力物理
        private float currentPushForce = 0f;
        private float lastLaserHitTime = 0f;
        private bool isReceivingLaserHit = false;
        private const float laserHitTimeout = 0.15f;
        // Press 角力推力累加
        private float accumulatedPushForceThisTick = 0f;
        private bool pushForceUpdatedThisTick = false;
        // Press 角力摩擦伤害
        private float clashTimer = 0f;
        private bool isFrictionDamageActive = false;
        private float frictionDamageAccumulator = 0f;
        // 战术召唤
        private float continuousDamageTimer = 0f;
        private bool tacticalSummonCooldown = false;
        private const float TACTICAL_SUMMON_THRESHOLD = 5f;
        private const float TACTICAL_SUMMON_CD = 10f;
        // Press 过载
        private float pressOverloadDamage = 0f;
        private float pressOverloadTimer = 0f;
        // 污秽球管理
        private  System.Collections.Generic.List<BossPollutionProjectile>
activePollutionBalls =
```

```
            new
System.Collections.Generic.List<BossPollutionProjectile>();
        // 短僵直标记
        private float stunDurationOverride = -1f;
        // 协程引用
        private Coroutine stateCoroutine;
        // 连续受伤检测
        private float lastDamageTime = 0f;
        private const float DAMAGE_GAP_THRESHOLD = 0.5f;
        // ═══════════════════════════════════════════
        // 【新增】冰冻与霸体系统
        // ═══════════════════════════════════════════
        // 冰冻累积
        private float frostExposureTime = 0f;          // 累计照射时间
        private const float FROST_EXPOSURE_RESET_DELAY = 0.3f; // 停止照射
后重置延迟
        private float frostExposureResetTimer = 0f;
        // 控制递减
        private int controlStack = 0;                      // 连续控制次数
(0-3)
        // 霸体状态
        private bool isUnstoppable = false;             // 是否处于霸体
        private float unstoppableTimer = 0f;            // 霸体剩余时间
        // 冰冻状态
        private bool isFrozen = false;                  // 是否冰冻中
        private float frozenTimer = 0f;                 // 冰冻剩余时间
        private Vector2 velocityBeforeFrozen;           // 冰冻前的速度
        private FrostDebuff frostDebuff;  // FrostDebuff 组件引用
        // ═══ 狂暴状态 ═══
        private bool hasTriggeredEnrage = false;  // 是否已触发过狂暴演出（防
止重复触发）
        private bool isEnrageEffectActive = false; // 狂暴红光效果是否激活
        // 缓存
        private ShieldController cachedShieldController;
#if DOTWEEN
        private Tweener moveTweener;
        private Tweener shakeTweener;
#endif
        // ═══════════════════════════════════════════
        // 属性
        // ═══════════════════════════════════════════
        /// <summary>Boss 配置（供 BossHealth 读取）</summary>
        public BossConfig Config => config;
        /// <summary>所有身体渲染器（供 BossHealth 使用受击效果）</summary>
        public SpriteRenderer[] BodyRenderers => bodyRenderers;
        /// <summary>原始颜色缓存（供 BossHealth 使用）</summary>
        public Color[] OriginalColors => originalColors;
        /// <summary>当前状态</summary>
        public BossState CurrentState => currentState;
        /// <summary>是否处于 Charge 蓄力阶段（可被 Impact Lv.5 打断）</summary>
```

```
        public bool IsInChargeTelegraph => currentState == BossState.Charge
&& !chargeInterrupted && !isUnstoppable;
        /// <summary>是否正在 Press 碾压中（可以被推）</summary>
        public bool IsPressing => currentState == BossState.Press &&
isPressing;
        /// <summary>血量百分比</summary>
        public float HealthPercent => bossHealth != null ?
bossHealth.HealthPercent : 1f;
        /// <summary>是否狂暴</summary>
        public bool IsEnraged => HealthPercent <= (config != null ?
config.rageHealthThreshold : 0.3f);
        /// <summary>【新增】是否处于霸体状态</summary>
        public bool IsUnstoppable => isUnstoppable;
        /// <summary>【新增】是否处于冰冻状态</summary>
        public bool IsFrozen => currentState == BossState.Frozen;
        /// <summary>【新增】当前控制递减层数</summary>
        public int ControlStack => controlStack;
        // ═══════════════════════════════════════════════
        // Unity 生命周期
        // ═══════════════════════════════════════════════
        private void Awake()
        {
            rb = GetComponent<Rigidbody2D>();
            bossHealth = GetComponent<BossHealth>();
            if (rb != null)
            {
                rb.gravityScale = 0f;
                rb.constraints = RigidbodyConstraints2D.FreezeRotation;
            }
        }
        private void Start()
        {
            InitializePositions();
            CalculateScreenBounds();
            CacheOriginalColors();
            // 【新增】初始化 FrostDebuff 并设置渲染器
            frostDebuff = GetComponent<FrostDebuff>();
            if (frostDebuff == null)
            {
                frostDebuff = gameObject.AddComponent<FrostDebuff>();
            }
            // 传入 Boss 的身体渲染器
            if (bodyRenderers != null && bodyRenderers.Length > 0)
            {
                frostDebuff.SetTargetRenderers(bodyRenderers);
            }
            summonCooldownTimer = config != null ? config.summonCooldown :
15f;
            summonCooldownReady = false;
            pollutionTimer = 0f;
```

```
            GameEvents.TriggerBossFightStart();
            ChangeState(BossState.Spawn);
        }
        private void Update()
        {
            UpdateSummonCooldown();
            UpdateContinuousDamageTimer();
            CheckTacticalSummon();
            // 【新增】更新霸体计时
            UpdateUnstoppableTimer();
            // 【新增】更新冰冻照射重置计时
            UpdateFrostExposureReset();
            CheckEnrageTrigger();
#if UNITY_EDITOR
            // 调试快捷键
            if (Input.GetKeyDown(KeyCode.K))
            {
                if (bossHealth != null)
                {
                    bossHealth.TakeCoreDamage(1000f,   transform.position,
false);
                }
            }
            if (Input.GetKeyDown(KeyCode.Alpha1))
            {
                ChangeState(BossState.Charge);
            }
            if (Input.GetKeyDown(KeyCode.Alpha2))
            {
                ChangeState(BossState.Press);
            }
            if (Input.GetKeyDown(KeyCode.Alpha3))
            {
                // 【新增】测试冰冻
                TryApplyFreeze(2f);
            }
            if (Input.GetKeyDown(KeyCode.Alpha4))
            {
                // 【新增】测试霸体
                EnterUnstoppable();
            }
#endif
        }
        private void FixedUpdate()
        {
            // Press 角力物理
            if (isPressing && isPressPhase3Active)
            {
                // 1. 施加下压力
                if (pressDownForce > 0)
```

```
                {
                    rb.AddForce(Vector2.down       *       pressDownForce,
ForceMode2D.Force);
                }
                // 2. 检查激光命中是否超时
                if (isReceivingLaserHit && Time.time - lastLaserHitTime >
laserHitTimeout)
                {
                    isReceivingLaserHit = false;
                    currentPushForce = 0f;
                }
                // 3. 施加激光推力（【修改】霸体时削弱）
                if (isReceivingLaserHit && currentPushForce > 0.01f)
                {
                    float actualPushForce = currentPushForce;
                    // 霸体期间推力削弱
                    if (isUnstoppable)
                    {
                        float    multiplier    =    config    !=    null    ?
config.unstoppablePushMultiplier : 0.3f;
                        actualPushForce *= multiplier;
                    }
                    rb.AddForce(Vector2.up       *       actualPushForce,
ForceMode2D.Force);
                    if (BattleStatistics.Instance != null)
                    {
                        BattleStatistics.Instance.MarkBossBeingPushed();
                    }
                }
                // 4. 摩擦伤害检测
                UpdateFrictionDamage();
                if (showDebugInfo && Time.frameCount % 30 == 0)
                {
                    float netForce = currentPushForce - pressDownForce;
                    GameLogger.Log($"[BossController]   角   力   :   下   压
={pressDownForce:F0}, 上推={currentPushForce:F0}, 净力={netForce:F0}, Y 速
度={rb.velocity.y:F2}, 霸体={isUnstoppable}");
                }
            }
        }
        // ═══════════════════════════════════════════════
        // 【新增】冰冻与霸体系统 - 公共接口
        // ═══════════════════════════════════════════════
        /// <summary>
        /// 累加 Frost 照射时间（由 LaserController 调用）
        /// </summary>
        public void AddFrostExposureTime(float deltaTime)
        {
            // 霸体期间不累积
            if (isUnstoppable) return;
```

```
            // 冰冻期间不累积
            if (currentState == BossState.Frozen) return;
            // 【新增】Press 角力期间不累积冰冻（但减速仍然生效）
            if (currentState == BossState.Press) return;
            frostExposureTime += deltaTime;
            frostExposureResetTimer = FROST_EXPOSURE_RESET_DELAY;
            // 检查是否达到冰冻阈值
            float threshold = config != null ?
config.bossFrostFreezeThreshold : 1.0f;
            if (frostExposureTime >= threshold)
            {
                float baseDuration = config != null ?
config.bossFrostFreezeDuration : 2.0f;
                TryApplyFreeze(baseDuration);
                frostExposureTime = 0f; // 重置累积
            }
        }
        /// <summary>
        /// 获取当前 Frost 照射时间
        /// </summary>
        public float GetFrostExposureTime() => frostExposureTime;
        /// <summary>
        /// 重置 Frost 照射时间
        /// </summary>
        public void ResetFrostExposureTime()
        {
            frostExposureTime = 0f;
        }
        /// <summary>
        /// 尝试应用冰冻效果（考虑控制递减）
        /// </summary>
        /// <param name="baseDuration">基础冰冻时长</param>
        /// <returns>是否成功应用冰冻</returns>
        public bool TryApplyFreeze(float baseDuration)
        {
            // 霸体期间免疫
            if (isUnstoppable)
            {
                if (showDebugInfo)
                {
                    GameLogger.Log("[BossController]   冰冻被霸体免疫！");
                }
                ShowStatusText(BossStatusTextType.Unstoppable);
                return false;
            }
            // 【新增】Press 角力期间免疫冰冻
            if (currentState == BossState.Press)
            {
                if (showDebugInfo)
```

```
                GameLogger.Log("[BossController]   冰冻被角力状态免疫！
");
                return false;
            }
            // 已经冰冻中
            if (currentState == BossState.Frozen)
            {
                return false;
            }
            // 计算控制递减后的实际时长
            float actualDuration = CalculateControlDuration(baseDuration);
            if (actualDuration <= 0f)
            {
                // 触发霸体
                EnterUnstoppable();
                return false;
            }
            // 应用冰冻
            ApplyFreeze(actualDuration);
            return true;
        }
        /// <summary>
        /// 尝试应用僵直效果（考虑控制递减）
        /// </summary>
        public bool TryApplyStun(float baseDuration)
        {
            // 霸体期间免疫
            if (isUnstoppable)
            {
                if (showDebugInfo)
                {
                    GameLogger.Log("[BossController]   僵直被霸体免疫！");
                }
                ShowStatusText(BossStatusTextType.Unstoppable);
                return false;
            }
            // 计算控制递减
            float actualDuration = CalculateControlDuration(baseDuration);
            if (actualDuration <= 0f)
            {
                EnterUnstoppable();
                return false;
            }
            // 应用僵直
            stunDurationOverride = actualDuration;
            ChangeState(BossState.Stun);
            // 增加控制计数
            controlStack++;
            if (showDebugInfo)
            {
```

```
                GameLogger.Log($"[BossController]     僵 直 生 效 ！ 时 长 :
{actualDuration:F2}s, 控制层数: {controlStack}");
            }
            return true;
        }
        /// <summary>
        /// 计算控制递减后的实际持续时间
        /// </summary>
        private float CalculateControlDuration(float baseDuration)
        {
            int     triggerCount     =     config     !=     null     ?
config.unstoppableTriggerCount : 3;
            if (controlStack >= triggerCount)
            {
                // 超过阈值，触发霸体
                return 0f;
            }
            float multiplier = 1f;
            switch (controlStack)
            {
                case 0:
                    multiplier = 1f; // 100%
                    break;
                case 1:
                    multiplier = config != null ? config.controlDiminish2nd :
0.5f; // 50%
                    break;
                case 2:
                    multiplier = config != null ? config.controlDiminish3rd :
0.25f; // 25%
                    break;
                default:
                    return 0f; // 触发霸体
            }
            // 狂暴状态下，控制效果持续时间额外减半
            if (IsEnraged && config != null)
            {
                multiplier *= config.rageControlDurationMultiplier;
            }
            return baseDuration * multiplier;
        }
        /// <summary>
        /// 进入霸体状态
        /// </summary>
        public void EnterUnstoppable()
        {
            if (isUnstoppable) return;
            isUnstoppable = true;
            unstoppableTimer = config != null ? config.unstoppableDuration :
6f;
```

```
            controlStack = 0; // 重置控制计数
            // 视觉效果：全身发红
            if (redBodyEffect != null)
            {
                redBodyEffect.SetActive(true);
            }
            // 显示飘字
            ShowStatusText(BossStatusTextType.Unstoppable);
            // 震动反馈
            if (CameraShake.Instance != null)
            {
                CameraShake.Instance.Shake(0.5f, 0.3f);
            }
            // 播放音效（可选）
            if (AudioManager.Instance != null)
            {
                AudioManager.Instance.PlayBossRoar();
            }
            if (showDebugInfo)
            {
                GameLogger.Log($"[BossController]    进入霸体状态！持续
{unstoppableTimer:F1}s");
            }
        }
        /// <summary>
        /// 退出霸体状态
        /// </summary>
        private void ExitUnstoppable()
        {
            if (!isUnstoppable) return;
            isUnstoppable = false;
            unstoppableTimer = 0f;
            controlStack = 0; // 重置控制计数
            // 关闭红色特效
            if (redBodyEffect != null)
            {
                redBodyEffect.SetActive(false);
            }
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController]   霸体状态结束！控制计数已
重置");
            }
        }
        /// <summary>
        /// 更新霸体计时器
        /// </summary>
        private void UpdateUnstoppableTimer()
        {
            if (!isUnstoppable) return;
```

```
            unstoppableTimer -= Time.deltaTime;
            if (unstoppableTimer <= 0f)
            {
                ExitUnstoppable();
            }
        }
        /// <summary>
        /// 更新冰冻照射重置计时
        /// </summary>
        private void UpdateFrostExposureReset()
        {
            if (frostExposureResetTimer > 0f)
            {
                frostExposureResetTimer -= Time.deltaTime;
                if (frostExposureResetTimer <= 0f && frostExposureTime > 0f)
                {
                    // 停止照射一段时间后重置累积
                    frostExposureTime = 0f;
                    if (showDebugInfo)
                    {
                        GameLogger.Log("[BossController]    冰冻累积已重置
（照射中断）");
                    }
                }
            }
        }
        /// <summary>
        /// 应用冰冻效果
        /// </summary>
        private void ApplyFreeze(float duration)
        {
            // 增加控制计数
            controlStack++;
            // 记录冰冻前状态
            stateBeforeFrozen = currentState;
            frozenTimer = duration;
            // 【修改】使用 FrostDebuff 处理视觉效果
            if (frostDebuff != null)
            {
                frostDebuff.ApplyFreeze(duration);
            }
            // 切换到冰冻状态
            ChangeState(BossState.Frozen);
            if (showDebugInfo)
            {
                GameLogger.Log($"[BossController]     冰冻生效！时长：
{duration:F2}s, 控制层数: {controlStack}");
            }
        }
        // ═══════════════════════════════════════════════
```

```
        // 初始化
        // ═══════════════════════════════════════════════════
        private void InitializePositions()
        {
            spawnPosition = new Vector3(0f, 10f, 0f);
            transform.position = spawnPosition;
            float anchorY = config != null ? config.battleAnchorY : 3.0f;
            battleAnchorPosition = new Vector3(0f, anchorY, 0f);
        }
        private void CalculateScreenBounds()
        {
            Camera cam = Camera.main;
            if (cam == null) return;
            float height = cam.orthographicSize * 2f;
            float width = height * cam.aspect;
            float     rangePercent     =     config     !=     null     ?
config.idleMoveRangePercent : 0.8f;
            screenMinX = -width * 0.5f * rangePercent;
            screenMaxX = width * 0.5f * rangePercent;
        }
        private void CacheOriginalColors()
        {
            if (bodyRenderers == null || bodyRenderers.Length == 0)
            {
                bodyRenderers = GetComponentsInChildren<SpriteRenderer>();
            }
            originalColors = new Color[bodyRenderers.Length];
            for (int i = 0; i < bodyRenderers.Length; i++)
            {
                if (bodyRenderers[i] != null)
                {
                    originalColors[i] = bodyRenderers[i].color;
                }
            }
        }
        // ═══════════════════════════════════════════════════
        // 状态机
        // ═══════════════════════════════════════════════════
        private void ChangeState(BossState newState)
        {
            if (showDebugInfo)
            {
                GameLogger.Log($"[BossController] 状态切换: {currentState}
-> {newState}");
            }
            ExitState(currentState);
            currentState = newState;
            // 【新增】上报 Boss 状态变化

BattleStatistics.Instance?.RecordBossPhase(newState.ToString());
```

```
            EnterState(newState);
        }
        private void ExitState(BossState state)
        {
            if (stateCoroutine != null)
            {
                StopCoroutine(stateCoroutine);
                stateCoroutine = null;
            }
            if (state == BossState.Charge)
            {
                chargeInterrupted = false;
                if    (redBodyEffect    !=    null    &&    !isUnstoppable
&& !isEnrageEffectActive)
                {
                    redBodyEffect.SetActive(false);
                }
            }
            else if (state == BossState.Press)
            {
                isPressing = false;
                accumulatedPushForce = Vector2.zero;
                isBeingPushed = false;
                // 霸体或狂暴时保持红光
                if    (redBodyEffect    !=    null    &&    !isUnstoppable
&& !isEnrageEffectActive)
                {
                    redBodyEffect.SetActive(false);
                }
            }
            else if (state == BossState.Frozen)
            {
                // 【新增】退出冰冻状态时恢复
                isFrozen = false;
                SetBodyFrozenVisual(false);
            }
#if DOTWEEN
            if   (moveTweener   !=   null   &&   moveTweener.IsActive())
moveTweener.Kill();
            if   (shakeTweener   !=   null   &&   shakeTweener.IsActive())
shakeTweener.Kill();
#endif
        }
        private void EnterState(BossState state)
        {
            switch (state)
            {
                case BossState.Spawn:
                    stateCoroutine = StartCoroutine(SpawnRoutine());
                    break;
```

```
                case BossState.Idle:
                    EnterIdle();
                    break;
                case BossState.Summon:
                    BattleStatistics.Instance?.RecordBossSkill("summon");
                    stateCoroutine = StartCoroutine(SummonRoutine());
                    break;
                case BossState.Charge:
                    BattleStatistics.Instance?.RecordBossSkill("charge");
                    stateCoroutine = StartCoroutine(ChargeRoutine());
                    break;
                case BossState.Press:
                    BattleStatistics.Instance?.RecordBossSkill("press");
                    stateCoroutine = StartCoroutine(PressRoutine());
                    break;
                case BossState.Stun:
                    BattleStatistics.Instance?.RecordBossSkill("stun");
                    stateCoroutine = StartCoroutine(StunRoutine());
                    break;
                case BossState.Frozen:
                    stateCoroutine = StartCoroutine(FrozenRoutine());
                    break;
            }
        }
        // ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
        // 【新增】State: Frozen (冰冻)
        // ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
        private IEnumerator FrozenRoutine()
        {
            isFrozen = true;
            // 保存并停止速度
            velocityBeforeFrozen = rb.velocity;
            rb.velocity = Vector2.zero;
            // 显示飘字
            ShowStatusText(BossStatusTextType.Frozen);
            // 播放冰冻音效
            if (AudioManager.Instance != null)
            {
                AudioManager.Instance.PlayEnemyFreeze();
            }
            if (showDebugInfo)
            {
                GameLogger.Log($"[BossController]    进入冰冻状态！时长：
{frozenTimer:F2}s");
            }
            // 等待冰冻结束
            while (frozenTimer > 0f)
            {
                frozenTimer -= Time.deltaTime;
                yield return null;
```

```
            }
            // 冰冻结束
            isFrozen = false;
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController]   冰冻结束！返回 Idle");
            }
            // 检查是否触发霸体
            int     triggerCount     =     config     !=     null     ?
config.unstoppableTriggerCount : 3;
            if (controlStack >= triggerCount)
            {
                EnterUnstoppable();
            }
            // 返回 Idle（根据配置）
            bool returnToIdle = config != null ? config.frozenReturnToIdle :
true;
            if (returnToIdle)
            {
                ChangeState(BossState.Idle);
            }
            else
            {
                // 尝试恢复之前的状态（如果合理）
                if (stateBeforeFrozen == BossState.Idle ||
                    stateBeforeFrozen == BossState.Spawn ||
                    stateBeforeFrozen == BossState.Frozen)
                {
                    ChangeState(BossState.Idle);
                }
                else
                {
                    ChangeState(stateBeforeFrozen);
                }
            }
        }
        /// <summary>
        /// 设置冰冻视觉效果
        /// </summary>
        /// <summary>
        /// 设置冰冻视觉效果（冰冻结束时调用）
        /// </summary>
        private void SetBodyFrozenVisual(bool frozen)
        {
            // 冰冻视觉效果已由 FrostDebuff 管理
            // 这里只处理冰冻结束时重置 FrostDebuff
            if (!frozen && frostDebuff != null)
            {
                frostDebuff.ResetDebuff();
            }
```

```
        }
        // ═══════════════════════════════════════════════
        // State: Spawn (入场)
        // ═══════════════════════════════════════════════
        private IEnumerator SpawnRoutine()
        {
            float duration = config != null ? config.spawnDuration : 2.5f;
            if             (eyeController             !=             null)
eyeController.SetStateDirect(BossEyeState.Closed);
            if (redBodyEffect != null) redBodyEffect.SetActive(false);
            if (showDebugInfo)
            {
                GameLogger.Log($"[BossController] 入场: {spawnPosition} ->
{battleAnchorPosition}");
            }
#if DOTWEEN
            bool moveComplete = false;
            moveTweener = transform.DOMove(battleAnchorPosition, duration)
                .SetEase(Ease.OutQuad)
                .OnComplete(() => moveComplete = true);
            while (!moveComplete) yield return null;
#else
            float elapsed = 0f;
            Vector3 startPos = transform.position;
            while (elapsed < duration)
            {
                elapsed += Time.deltaTime;
                float t = 1f - Mathf.Pow(1f - elapsed / duration, 2f);
                transform.position        =        Vector3.Lerp(startPos,
battleAnchorPosition, t);
                yield return null;
            }
            transform.position = battleAnchorPosition;
#endif
            if (showDebugInfo) GameLogger.Log("[BossController] BOSS 咆哮!
");
            if (AudioManager.Instance != null)
            {
                AudioManager.Instance.PlayBossRoar();
            }
            float     shakeIntensity     =     config     !=     null     ?
config.spawnShakeIntensity : 0.5f;
            float     shakeDuration     =     config     !=     null     ?
config.spawnShakeDuration : 0.5f;
            if (CameraShake.Instance != null)
            {
                CameraShake.Instance.Shake(shakeIntensity,
shakeDuration);
            }
            yield return new WaitForSeconds(shakeDuration);
```

```
            ChangeState(BossState.Idle);
        }
        // ═══════════════════════════════════════════════
        // State: Idle (待机/游走)
        // ═══════════════════════════════════════════════
        private void EnterIdle()
        {
            if (eyeController != null) eyeController.Close();
            currentIdleDuration = config != null ?
config.GetIdleDuration(HealthPercent) : Random.Range(3f, 5f);
            idleTimer = 0f;
            SetNextIdleMoveTarget();
            stateCoroutine = StartCoroutine(IdleRoutine());
            if (showDebugInfo)
            {
                GameLogger.Log($"[BossController] 进入 Idle, 时长:
{currentIdleDuration:F1}s, 狂暴: {IsEnraged}");
            }
        }
        private IEnumerator IdleRoutine()
        {
            float moveSpeed = config != null ?
config.GetMoveSpeed(HealthPercent) : 1.5f;
            pollutionTimer = 0f;
            float pollutionInterval = config != null ?
config.pollutionInterval : 4f;
            while (idleTimer < currentIdleDuration)
            {
                idleTimer += Time.deltaTime;
                if (summonCooldownReady)
                {
                    if (showDebugInfo) GameLogger.Log("[BossController] ⏰
召唤冷却完成！打断 Idle 进入 Summon");
                    ChangeState(BossState.Summon);
                    yield break;
                }
                pollutionTimer += Time.deltaTime;
                if (pollutionTimer >= pollutionInterval)
                {
                    pollutionTimer = 0f;
                    FirePollutionProjectile();
                }
                float currentX = transform.position.x;
                float newX = Mathf.MoveTowards(currentX, idleMoveTargetX,
moveSpeed * Time.deltaTime);
                transform.position = new Vector3(newX, transform.position.y,
transform.position.z);
                if (Mathf.Abs(newX - idleMoveTargetX) < 0.1f)
                {
                    SetNextIdleMoveTarget();
```

```
            }
            yield return null;
        }
        ChooseActiveSkill();
    }
    private void SetNextIdleMoveTarget()
    {
        idleMoveTargetX = Random.Range(screenMinX, screenMaxX);
    }
    private void ChooseActiveSkill()
    {
        bool useCharge = Random.value < 0.5f;
        if (useCharge)
        {
            if (showDebugInfo) GameLogger.Log("[BossController]   选择
技能: Charge (野蛮冲撞 - 快招)");
            ChangeState(BossState.Charge);
        }
        else
        {
            if (showDebugInfo) GameLogger.Log("[BossController]   选择
技能: Press (重力碾压 - 慢招)");
            ChangeState(BossState.Press);
        }
    }
    // ═══════════════════════════════════════════════
    // Summon 冷却管理
    // ═══════════════════════════════════════════════
    private void UpdateSummonCooldown()
    {
        if (summonCooldownTimer > 0)
        {
            summonCooldownTimer -= Time.deltaTime;
            if (summonCooldownTimer <= 0)
            {
                summonCooldownReady = true;
            }
        }
    }
    private void ResetSummonCooldown()
    {
        float     cooldown     =     config     !=     null     ?
config.GetSummonCooldown(HealthPercent) : 15f;
        summonCooldownTimer = cooldown;
        summonCooldownReady = false;
        if (showDebugInfo)
        {
            GameLogger.Log($"[BossController]   召 唤 冷 却 重 置 :
{cooldown}s");
        }
```

```
        }
        // ═══════════════════════════════════════════════
        // State: Summon (召唤爪牙) - 被动技能
        // ═══════════════════════════════════════════════
        private IEnumerator SummonRoutine()
        {
            float duration = config != null ? config.summonDuration : 1.0f;
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController]    召唤爪牙！身体收缩震
动...");
            }
            float blinkDuration = config != null ? config.blinkDuration :
0.75f;
            if (eyeController != null)
            {
                eyeController.Blink(blinkDuration);
            }
#if DOTWEEN
            if (bodyTransform != null)
            {
                float     intensity     =     config     !=     null     ?
config.summonShakeIntensity : 0.1f;
                shakeTweener   =   bodyTransform.DOShakePosition(duration,
intensity, 20, 90, false, true);
            }
#endif
            yield return new WaitForSeconds(duration);
            SpawnRushers();
            ResetSummonCooldown();
            ChangeState(BossState.Idle);
        }
        private void SpawnRushers()
        {
            if (EnemyPoolManager.Instance == null) return;
            int perSide = config != null ? config.summonRusherPerSide : 2;
            float     speedBonus     =     config     !=     null     ?
config.GetRusherSpeedBonus(HealthPercent) : 1f;
            Camera cam = Camera.main;
            if (cam == null) return;
            float halfWidth = cam.orthographicSize * cam.aspect;
            float spawnY = Random.Range(-2f, 2f);
            for (int i = 0; i < perSide; i++)
            {
                Vector3 pos = new Vector3(-halfWidth - 1f - i * 0.5f, spawnY
+ i * 0.3f, 0f);
                var                          enemy                          =
EnemyPoolManager.Instance.Spawn(EnemyType.Rusher, pos);
                if (enemy != null && speedBonus > 1f)
                {
```

```
                enemy.SetWaveModifiers(new DifficultyModifiers
                {
                    hpMultiplier = 1f,
                    speedMultiplier = speedBonus,
                    massMultiplier = 1f,
                    damageMultiplier = 1f
                });
            }
        }
        for (int i = 0; i < perSide; i++)
        {
            Vector3 pos = new Vector3(halfWidth + 1f + i * 0.5f, spawnY
- i * 0.3f, 0f);
            var                         enemy                         =
EnemyPoolManager.Instance.Spawn(EnemyType.Rusher, pos);
            if (enemy != null && speedBonus > 1f)
            {
                enemy.SetWaveModifiers(new DifficultyModifiers
                {
                    hpMultiplier = 1f,
                    speedMultiplier = speedBonus,
                    massMultiplier = 1f,
                    damageMultiplier = 1f
                });
            }
        }
        if (showDebugInfo)
        {
            GameLogger.Log($"[BossController] 生成 {perSide * 2} 只
Rusher (狂暴: {IsEnraged})");
        }
    }
    // ═══════════════════════════════════════════════
    // 战术召唤
    // ═══════════════════════════════════════════════
    private void UpdateContinuousDamageTimer()
    {
        if (Time.time - lastDamageTime > DAMAGE_GAP_THRESHOLD)
        {
            continuousDamageTimer = 0f;
        }
    }
    public void OnDamageReceived()
    {
        lastDamageTime = Time.time;
        continuousDamageTimer += Time.deltaTime;
    }
    private void CheckTacticalSummon()
    {
        if (currentState != BossState.Idle) return;
```

```
            if (tacticalSummonCooldown) return;
            if (continuousDamageTimer >= TACTICAL_SUMMON_THRESHOLD)
            {
                if (showDebugInfo)
                {
                    GameLogger.Log("[BossController] ⚡ 战术召唤触发！玩家输出太安逸了！");
                }
                continuousDamageTimer = 0f;
                tacticalSummonCooldown = true;
                StartCoroutine(TacticalSummonCooldownRoutine());
                ChangeState(BossState.Summon);
            }
        }
        private IEnumerator TacticalSummonCooldownRoutine()
        {
            yield return new WaitForSeconds(TACTICAL_SUMMON_CD);
            tacticalSummonCooldown = false;
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController] 战术召唤冷却结束");
            }
        }
        // ═══════════════════════════════════════════════════
        // Pollution (污秽喷吐)
        // ═══════════════════════════════════════════════════
        private void FirePollutionProjectile()
        {
            GameObject prefab = config != null ? config.pollutionProjectilePrefab : null;
            if (prefab == null)
            {
                if (showDebugInfo) GameLogger.LogWarning("[BossController] Pollution Prefab 未设置！");
                return;
            }
            int maxCount = config != null ? config.pollutionMaxCount : 3;
            int burstCount = config != null ? config.GetPollutionBurstCount(HealthPercent) : 3;
            float spreadAngle = config != null ? config.pollutionSpreadAngle : 30f;
            activePollutionBalls.RemoveAll(b => b == null || b.IsDestroyed);
            int available = maxCount - activePollutionBalls.Count;
            int toSpawn = Mathf.Min(burstCount, available);
            if (toSpawn <= 0)
            {
                if (showDebugInfo) GameLogger.Log("[BossController] 污秽球已达上限，跳过喷吐");
                return;
            }
```

```
            Vector3 spawnPos = transform.position;
            Vector2 baseDir = Vector2.down;
            float startAngle = -spreadAngle * 0.5f;
            float angleStep = toSpawn > 1 ? spreadAngle / (toSpawn - 1) : 0f;
            for (int i = 0; i < toSpawn; i++)
            {
                float angle = startAngle + angleStep * i;
                Vector2 dir = Quaternion.Euler(0, 0, angle) * baseDir;
                GameObject    go    =    Instantiate(prefab,    spawnPos,
Quaternion.identity);
                var ball = go.GetComponent<BossPollutionProjectile>();
                if (ball != null)
                {
                    // V3.0: 初始化带物理参数
                    if (config != null)
                    {
                        ball.InitializeV3(
                            config.pollutionSpeed,
                            config.pollutionTurnSpeed,
                            config.pollutionShieldDamage,
                            config.pollutionLifetime,
                            config.pollutionBallHP,
                            config.pollutionBallMass,
                            this
                        );
                        // 设置初始方向（计算角度偏移）
                        float  angleOffset  =  Mathf.Atan2(dir.x,  -dir.y)  *
Mathf.Rad2Deg;
                        ball.SetInitialDirection(angleOffset);
                    }
                    RegisterPollutionBall(ball);
                }
            }
            if (showDebugInfo)
            {
                GameLogger.Log($"[BossController] 喷射 {toSpawn} 个污秽球
");
            }
        }
        // ═══════════════════════════════════════════════════
        // State: Charge (野蛮冲撞) - 主动技能 A【快招】
        // ═══════════════════════════════════════════════════
        private IEnumerator ChargeRoutine()
        {
            chargeInterrupted = false;
            chargeHitCount = 0;
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController]   Charge Phase 1: 蓄力!");
            }
```

```
            if (redBodyEffect != null && !isUnstoppable)
            {
                redBodyEffect.SetActive(true);
            }
            if (eyeController != null) eyeController.Open();
            float    telegraphDuration    =    config    !=    null    ?
config.chargeTelegraphDuration : 1.0f;
            float    windupDistance    =    config    !=    null    ?
config.chargeWindupDistance : 0.5f;
            // 播放预警音效
            if (AudioManager.Instance != null)
            {
                AudioManager.Instance.PlayBossChargeWarning();
            }
            // 身体后退（像拉弓）
            Vector3  windupPos  =  transform.position  +  Vector3.up  *
windupDistance;
        #if DOTWEEN
            transform.DOMove(windupPos, 0.2f).SetEase(Ease.OutQuad);
        #else
            float windupTime = 0f;
            Vector3 startPos = transform.position;
            while (windupTime < 0.2f)
            {
                windupTime += Time.deltaTime;
                transform.position  =  Vector3.Lerp(startPos,  windupPos,
windupTime / 0.2f);
                yield return null;
            }
        #endif
            float elapsed = 0f;
            while (elapsed < telegraphDuration)
            {
                if (chargeInterrupted)
                {
                    OnChargeInterrupted();
                    yield break;
                }
                elapsed += Time.deltaTime;
                yield return null;
            }
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController]   Charge Phase 2: 冲锋!");
            }
            if (AudioManager.Instance != null)
            {
                AudioManager.Instance.PlayBossDash();
            }
            float speedMultiplier = GetChargeSpeedMultiplier();
```

```
            float baseDashDuration = config != null ?
config.chargeDashDuration : 0.3f;
            float dashDuration = baseDashDuration / speedMultiplier;
            float targetY = config != null ? config.chargeTargetY : -10f;
            Vector3 dashTarget = new Vector3(transform.position.x, targetY,
transform.position.z);
        #if DOTWEEN
            bool dashComplete = false;
            moveTweener = transform.DOMove(dashTarget, dashDuration)
                .SetEase(Ease.InQuad)
                .OnComplete(() => dashComplete = true);
            while (!dashComplete) yield return null;
        #else
            float dashElapsed = 0f;
            Vector3 dashStart = transform.position;
            while (dashElapsed < dashDuration)
            {
                dashElapsed += Time.deltaTime;
                float t = dashElapsed / dashDuration;
                t = t * t;
                transform.position = Vector3.Lerp(dashStart, dashTarget,
t);
                yield return null;
            }
            transform.position = dashTarget;
        #endif
            OnChargeHitPlayer();
        }
        public void OnHitReceived()
        {
            if (currentState != BossState.Charge) return;
            if (chargeInterrupted) return;
            chargeHitCount++;
            int threshold = config != null ?
config.chargeHitCountThreshold : 30;
            if (chargeHitCount >= threshold)
            {
                chargeInterrupted = true;
                if (showDebugInfo)
                {
                    GameLogger.Log($"[BossController] 频率打断！受击次数:
{chargeHitCount}");
                }
            }
        }
        public void InterruptCharge()
        {
            // 【修改】霸体期间免疫打断
            if (isUnstoppable)
            {
```

```
                if (showDebugInfo)
                {
                    GameLogger.Log("[BossController] ⚡ Charge 打断被霸体免
疫！");
                }
                ShowStatusText(BossStatusTextType.Unstoppable);
                return;
            }
            if (currentState != BossState.Charge) return;
            if (chargeInterrupted) return;
            chargeInterrupted = true;
        }
        private void OnChargeHitPlayer()
        {
            float damage = config != null ? config.chargeHitDamage : 300f;
            ApplyDamageToPlayer(damage,
PlayerDamageSource.BossCollision);
            if (showDebugInfo) GameLogger.Log($"[BossController]   Charge
撞击玩家！伤害: {damage}");
            float     shakeIntensity     =     config     !=     null     ?
config.chargeHitShakeIntensity : 0.8f;
            float     shakeDuration     =     config     !=     null     ?
config.chargeHitShakeDuration : 0.3f;
            if (CameraShake.Instance != null)
            {
                CameraShake.Instance.ImpactShake(Vector2.down,
shakeIntensity, shakeDuration);
            }
            StartCoroutine(ChargeBounceBackRoutine());
        }
        private IEnumerator ChargeBounceBackRoutine()
        {
            float      duration      =      config      !=      null      ?
config.chargeBounceBackDuration : 0.5f;
#if DOTWEEN
            yield return transform.DOMove(battleAnchorPosition, duration)
                .SetEase(Ease.OutQuad)
                .WaitForCompletion();
#else
            float elapsed = 0f;
            Vector3 start = transform.position;
            while (elapsed < duration)
            {
                elapsed += Time.deltaTime;
                transform.position          =          Vector3.Lerp(start,
battleAnchorPosition, elapsed / duration);
                yield return null;
            }
            transform.position = battleAnchorPosition;
#endif
```

```
            if (eyeController != null) eyeController.Close();
            yield return new WaitForSeconds(1.0f);
            ChangeState(BossState.Idle);
        }
        private void OnChargeInterrupted()
        {
            if (showDebugInfo)
                GameLogger.Log("[BossController]  Charge 蓄力被打断!进入僵直! ");
            ShowStatusText(BossStatusTextType.Interrupted);
            if (redBodyEffect != null && !isUnstoppable)
            {
                redBodyEffect.SetActive(false);
            }
            // 【修改】使用控制递减的僵直
            float baseDuration = config != null ? config.chargeInterruptStunDuration : 3.0f;
            // 修复：如果晕眩失败（被霸体免疫），必须强制切换状态，否则会卡在 Charge 状态
            if (!TryApplyStun(baseDuration))
            {
                if (showDebugInfo) GameLogger.Log("[BossController] 霸体免疫打断僵直，强制返回 Idle");
                ChangeState(BossState.Idle);
            }
        }
        // ═══════════════════════════════════════════════
        // State: Press (重力碾压) - 主动技能 B【慢招/角力】
        // ═══════════════════════════════════════════════
        private IEnumerator PressRoutine()
        {
            isPressing = false;
            accumulatedPushForce = Vector2.zero;
            clashTimer = 0f;
            isFrictionDamageActive = false;
            frictionDamageAccumulator = 0f;
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController]  Press Phase 1: 突进贴脸!（闭眼）");
            }
            if (AudioManager.Instance != null)
            {
                AudioManager.Instance.PlayBossDash();
            }
            if (eyeController != null) eyeController.Close();
            float jumpDuration = config != null ? config.pressJumpDuration : 0.5f;
            float hoverY = config != null ? config.pressHoverY : -5f;
            Vector3 hoverPosition = new Vector3(0f, hoverY, 0f);
```

```
#if DOTWEEN
            bool jumpComplete = false;
            moveTweener = transform.DOMove(hoverPosition, jumpDuration)
                .SetEase(Ease.OutExpo)
                .OnComplete(() => jumpComplete = true);
            while (!jumpComplete) yield return null;
#else
            float jumpElapsed = 0f;
            Vector3 jumpStart = transform.position;
            while (jumpElapsed < jumpDuration)
            {
                jumpElapsed += Time.deltaTime;
                float t = 1f - Mathf.Pow(1f - jumpElapsed / jumpDuration, 3f);
                transform.position = Vector3.Lerp(jumpStart, hoverPosition,
t);
                yield return null;
            }
            transform.position = hoverPosition;
#endif
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController]   Press Phase 2: 施压！眼
睛缓缓睁开...");
            }
            float     glareDuration     =     config     !=     null     ?
config.pressGlareDuration : 1.5f;
            if (redBodyEffect != null && !isUnstoppable)
            {
                redBodyEffect.SetActive(true);
            }
            if (eyeController != null)
            {
                eyeController.OpenSlowly(glareDuration);
            }
            yield return new WaitForSeconds(glareDuration);
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController] Press Phase 3: 开始碾压！
角力进行中...");
            }
            pressOverloadDamage = 0f;
            pressOverloadTimer = 0f;
            isPressing = true;
            isPressPhase3Active = true;
            pressDownForce       =       config       !=       null       ?
config.GetPressForce(HealthPercent) : 100f;
            float safeLineY = config != null ? config.pressSafeLineY : 3.5f;
            float hitLineY = config != null ? config.pressHitLineY : -10f;
            float maxDuration = config != null ? config.pressMaxDuration :
15f;
```

```
            float pressTimer = 0f;
            while (pressTimer < maxDuration)
            {
                pressTimer += Time.deltaTime;
                clashTimer += Time.deltaTime;
                accumulatedPushForceThisTick = 0f;
                pushForceUpdatedThisTick = false;
                if (transform.position.y >= safeLineY)
                {
                    if (showDebugInfo) GameLogger.Log("[BossController] ✅
玩家推回 Boss！角力胜利！");
                    OnPressCountered();
                    yield break;
                }
                if (transform.position.y <= hitLineY)
                {
                    if (showDebugInfo) GameLogger.Log("[BossController] ❌
Boss 碾压成功！玩家失败！");
                    OnPressHitPlayer();
                    yield break;
                }
                float    maxClashTime    =    config    !=    null    ?
config.maxClashDuration : 6f;
                if (clashTimer >= maxClashTime)
                {
                    if (showDebugInfo) GameLogger.Log("[BossController] ⏰
角力超时！Boss 疲劳撤退！");
                    OnPressExhausted();
                    yield break;
                }
                yield return null;
            }
            if (showDebugInfo) GameLogger.Log("[BossController] ⏰ Press
超时！自动结束");
            OnPressExhausted();
        }
        private void UpdateFrictionDamage()
        {
            float triggerY = config != null ? config.frictionTriggerY :
-8.5f;
            if (transform.position.y <= triggerY)
            {
                if (!isFrictionDamageActive)
                {
                    isFrictionDamageActive = true;
                    GameEvents.TriggerBossFrictionStart();
                    if (showDebugInfo)
                    {
                        GameLogger.Log($"[BossController]   摩擦伤害开始！
Y={transform.position.y:F2}");
```

```
                }
            }
            float dps = config != null ? config.frictionDamagePerSecond :
50f;
            frictionDamageAccumulator += dps * Time.fixedDeltaTime;
            if (frictionDamageAccumulator >= 1f)
            {
                int                    damage                    =
Mathf.FloorToInt(frictionDamageAccumulator);
                frictionDamageAccumulator -= damage;
                ApplyDamageToPlayer(damage,
PlayerDamageSource.BossFriction);
            }
        }
        else if (isFrictionDamageActive)
        {
            isFrictionDamageActive = false;
            GameEvents.TriggerBossFrictionEnd();
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController]   摩擦伤害结束");
            }
        }
        float window = config != null ? config.pressOverloadWindow :
1.5f;
        pressOverloadTimer += Time.fixedDeltaTime;
        if (pressOverloadTimer >= window)
        {
            pressOverloadDamage = 0f;
            pressOverloadTimer = 0f;
        }
    }
    public void RecordPressOverloadDamage(float damage)
    {
        if (!isPressing) return;
        pressOverloadDamage += damage;
        float     threshold     =     config     !=     null     ?
config.pressOverloadDamageThreshold : 2000f;
        if (pressOverloadDamage >= threshold)
        {
            if (showDebugInfo)
            {
                GameLogger.Log($"[BossController]   Press 过载!累计伤害
{pressOverloadDamage:F0} >= {threshold:F0}");
            }
            OnPressOverload();
        }
    }
    private void OnPressOverload()
    {
```

```
            isPressPhase3Active = false;
            pressDownForce = 0f;
            currentPushForce = 0f;
            isReceivingLaserHit = false;
            isFrictionDamageActive = false;
            frictionDamageAccumulator = 0f;
            pressOverloadDamage = 0f;
            rb.velocity = Vector2.zero;
            isPressing = false;
            GameEvents.TriggerBossFrictionEnd();
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController]   Press 过载！Boss 核心过
热撤退！");
            }
            ShowStatusText(BossStatusTextType.Overload);
            float     shortDuration     =     config     !=     null     ?
config.shortStunDuration : 1.5f;
            if (!TryApplyStun(shortDuration))
            {
                if (showDebugInfo) GameLogger.Log("[BossController] 霸体免
疫过载僵直，强制返回 Idle");
                ChangeState(BossState.Idle);
            }
            EnterShortStun();
        }
        private void OnPressCountered()
        {
            isPressPhase3Active = false;
            pressDownForce = 0f;
            currentPushForce = 0f;
            isReceivingLaserHit = false;
            isFrictionDamageActive = false;
            frictionDamageAccumulator = 0f;
            rb.velocity = Vector2.zero;
            isPressing = false;
            GameEvents.TriggerBossFrictionEnd();
            if (showDebugInfo)
            {
                GameLogger.Log("[BossController] ✓ Press 被反推！玩家胜利！
");
```